

```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyStocks.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S10_Main";
    }
    /// <summary>
    /// S00_Boot scene host: register mock services -> run seven steps -> switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// </summary>
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null && !op.isDone) yield return null;
        }
    }
}

// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.CrazyStocks.Boot
{
    /// <summary>
```

```
/// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
/// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &= Run(1, "Logger", LoggerStep);
        ok &= Run(2, "Time", TimeStep);
        ok &= Run(3, "Config", ConfigStep);
        ok &= Run(4, "Save", SaveStep);
        ok &= Run(5, "ILocale", LocaleStep);
        ok &= Run(6, "Ads.Preload", AdsStep);
        ok &= Run(7, "SwitchMain", () =&gt; _switchScene != null && _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 && !time.TimeCheated();
    }
    static bool ConfigStep()
    {

```

```
        var config = ServiceLocator.Get<IConfigService>();
        var update = config.CheckUpdate("mock://manifest");
        return update.NewVersion != null; // mock = no hot update
    }
    static bool SaveStep()
    {
        var save = ServiceLocator.Get<ISaveService>();
        save.Save("boot_probe", "{}");
        return !string.IsNullOrEmpty(save.Load("boot_probe"));
    }
    static bool LocaleStep()
    {
        var locale = ServiceLocator.Get<ILocaleService>();
        return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
    }
    static bool AdsStep()
    {
        ServiceLocator.Get<IAdsService>().Preload("boot_warmup");
        return true;
    }
}

// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyStocks.Boot
{
    /// <summary>
    /// Dev-time mock service set (SY MOCK scope). All seven boot steps depend on this set.
    /// </summary>
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet<ITimeService>(out _)) return; // idempotent: double register would throw
            ServiceLocator.Register<ITimeService>(new MockTimeService());
            ServiceLocator.Register<IConfigService>(new MockConfigService());
            ServiceLocator.Register<ISaveService>(new MockSaveService());
            ServiceLocator.Register<ILocaleService>(new MockLocaleService());
            ServiceLocator.Register<IAdsService>(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY MOCK)");
        }
    }

    public sealed class MockTimeService : ITimeService
    {

```

```

    public TimeSource Source => TimeSource.Mock;
    public long ClientNowMs() => DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// <summary></summary>SY MOCK: device time doubles as server time until the real chain lands.</summary>
    public long ServerNowMs() => ClientNowMs();
    public bool TimeCheated() => Math.Abs(ServerNowMs() - ClientNowMs()) > 120_000;
}

public sealed class MockConfigService : IConfigService
{
    public T GetTable<T>(string tableId) where T : class
        => throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable<T>(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        => new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) => "mock-0.0.1";
}

public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) => PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) => PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// <summary></summary>No cloud: local storage is the authority (zero-server doctrine).</summary>
    public SyncResult CloudSync() => new SyncResult(false, null);
}

public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// <summary></summary>Missing key falls back to the key itself with a warning.</summary>
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        => new FontSet("G10Main", "G10Main", "G10Main");
}

public sealed class MockAdsService : IAdsService
{
    public event Action<AdLifecycle> OnAdEvent;
    public Task<RewardResult> ShowRewarded(string pointId)
        => Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
    public void Preload(string pointId)
    {
        Logger.Info("ADS", "preload (mock): " + pointId);
    }
}

```

```

        OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
    }
}

// TradeArt.cs
// TradeArt.cs - the T019 art face (S2.5): the frozen closed set of the
// fifteen art-doc §7 stems (G10-A01..A15) as the single authoritative
// stem registry, plus null-safe lookup helpers. X027 iron rule:
// art-first, procedural fallback - every accessor returns null when the
// stem has not shipped and callers MUST treat null as "keep the
// procedural placeholder" so headless gates stay green on any project
// state (catalog absent, art pruned, name typos). Stems are frozen:
// adding or renaming one requires a §7 doc row first (the U024
// table-production law - nothing enters the project outside the table).
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
namespace Sy.CrazyStocks
{
    /// <summary>
    /// The S2.5 art stem registry (T019). The closed set mirrors
    /// the art asset demand doc section 7 rows G10-A01..A15 exactly;
    /// ArtCheck asserts the parity so a drift between doc and code
    /// fails the gate.
    /// </summary>
    public static class TradeArt
    {
        // ---- hall stages (A01/A02: the S10_Main day/close backdrops) ----
        public const string HallDay = "g10_hall_day";
        public const string HallClose = "g10_hall_close";
        // ---- hero portraits (A03/A04: the create-pick faces) ----
        public const string HeroMale = "g10_hero_m";
        public const string HeroFemale = "g10_hero_f";
        // ---- meme-stock icons (A05..A08: holdings/codex/event faces) ----
        public const string StockLeek = "g10_stock_leek";
        public const string StockPancake = "g10_stock_pancake";
        public const string StockAudio = "g10_stock_audio";
        public const string StockCola = "g10_stock_cola";
        // ---- trader portraits (A09..A12: the four frozen comedy faces) ----
        public const string TraderSuoha = "g10_trd_suoha";
        public const string TraderTaoding = "g10_trd_taoding";
        public const string TraderLaoli = "g10_trd_laoli";
        public const string TraderDama = "g10_trd_dama";
        // ---- ritual npc + ui containers (A13..A15) ----
        public const string NpcBell = "g10_npc_bell";
        public const string EventCardFrame = "g10_eventcard_frame";
        public const string UiKit = "g10_ui_kit";
        /// <summary>The frozen closed set (§7 parity face, 15 stems,
        /// ordinal sort so two builds land the same sequence).</summary>
        public static readonly string[] AllStems =

```

```

{
    EventCardFrame, HallClose, HallDay, HeroFemale, HeroMale,
    NpcBell, StockAudio, StockCola, StockLeek, StockPancake,
    TraderDama, TraderLaoli, TraderSuoha, TraderTaoding, Uikit,
};
/// <summary>Known comedy traders on the in-game roster map to the
/// four frozen §7 portraits; the rest return null (procedural
/// placeholder is the legal face for them until §7 grows rows).</summary>
static readonly Dictionary<string, string> TraderPortraitMap = new Dictionary<string, string>
{
    { "trd_all_in_bro",    TraderSuoha },    // the all-in attitude bro
    { "trd_top_escape_sis", TraderTaoding }, // the top-escape sister
    { "trd_counter_li",   TraderLaoli },    // the reverse-indicator Li
    { "trd_granny_info",  TraderDama },     // the square-dance info auntie
};
/// <summary>§7 portrait stem for a roster trader id (null = no
/// frozen portrait, procedural placeholder owns the face).</summary>
public static string TraderPortraitStem(string traderId)
    => traderId != null && TraderPortraitMap.TryGetValue(traderId, out var stem) ? stem : null;
/// <summary>The shipped art sprite for a stem (null on the
/// placeholder path - never throws, headless-safe).</summary>
public static Sprite Get(string stem) => ArtRuntime.Get(stem);
}
}
// TradeAudio.cs
// TradeAudio.cs - the T020 audio face: the stamp-to-stem map, the
// per-project AudioCatalog (the X027 art-catalog paradigm applied to
// audio), and a null-safe playback service. The catalog asset is built
// by the editor tool (AudioSetup) from Assets/_Game/Audio; runtime
// consumers MUST treat a missing catalog/clip as "skip silently" so
// headless checks stay green on any project state (G02Sfx headless law).
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.CrazyStocks
{
    /// <summary>
    /// Generated-audio catalog (T020, X027 paradigm): the AudioSetup editor
    /// tool scans Assets/_Game/Audio and serializes clip references here so
    /// runtime code resolves audio by filename stem. The asset lives at
    /// Assets/_Game/Resources/AudioCatalog.asset (Resources keeps runtime
    /// lookup build-safe with zero Addressables wiring for playtest slices).
    /// </summary>
    [CreateAssetMenu(fileName = "AudioCatalog", menuName = "Sy/CrazyStocks Audio Catalog")]
    public sealed class AudioCatalog : ScriptableObject
    {
        [Serializable]
        public sealed class Entry
        {
            public string key;        // filename stem, e.g. "g10_sfx_match"

```

```

        public AudioClip clip; // direct reference keeps builds self-contained
    }
    public List<Entry> entries = new List<Entry>();
    Dictionary<string, AudioClip> _index;
    void OnEnable()
    {
        // ScriptableObject instances can be revived with a stale cache;
        // drop it so the next Get() rebuilds from entries.
        _index = null;
    }
    public AudioClip Get(string key)
    {
        if (string.IsNullOrEmpty(key)) return null;
        if (_index == null)
        {
            _index = new Dictionary<string, AudioClip>(entries.Count, StringComparer.Ordinal);
            for (int i = 0; i < entries.Count; i++)
            {
                var e = entries[i];
                if (e != null && !string.IsNullOrEmpty(e.key) && e.clip != null && !_index.ContainsKey(e.key))
                    _index[e.key] = e.clip;
            }
        }
        return _index.TryGetValue(key, out var c) ? c : null;
    }
}
/// <summary>
/// Runtime lookup helper. Every consumer MUST treat null as "clip
/// missing -> skip" so headless checks stay green on any project state
/// (catalog absent, audio pruned, name typos).
/// </summary>
public static class AudioRuntime
{
    static AudioCatalog _catalog;
    public static AudioCatalog Catalog
    {
        get
        {
            if (_catalog == null) _catalog = Resources.Load<AudioCatalog>("AudioCatalog");
            return _catalog;
        }
    }
    public static AudioClip Get(string key)
    {
        var c = Catalog;
        return c != null ? c.Get(key) : null;
    }
}
/// <summary>
/// The audio face of the trading floor (T020): maps stage stamps,

```

```

/// market session marks, and named face cues to catalog stems, and
/// plays them null-safe on hidden hosts (never throws headless).
/// Stems are the delivered file stems from the G10 audio manifest
/// (S1A1/A2/A3 batches): two loop channels (main BGM + hall bed)
/// and twelve one-shots. The signature hook (earworm red line) is
/// baked into bgm_main and carried by the open/close/match family.
/// &lt;/summary&gt;
public static class TradeAudio
{
    // ---- delivered stems (the closed set, 14) ----
    public const string StemBgmMain = "g10_bgm_main";           // trading-day loop BGM (hook baked in)
    public const string StemAmbHall = "g10_amb_hall";           // trading-hall crowd bed
    public const string StemSfxMatch = "g10_sfx_match";          // fill chime (the typewriter-riff cousin)
    public const string StemSfxCoin = "g10_sfx_coin";            // coin family (fills + dividends)
    public const string StemSfxUpgrade = "g10_sfx_upgrade";      // trader upgrade burst
    public const string StemSfxOpen = "g10_sfx_open";            // opening gong (hook piece 1)
    public const string StemSfxClose = "g10_sfx_close";          // closing bell (hook piece 3)
    public const string StemSfxLimit = "g10_sfx_limit";          // limit-up/down drum
    public const string StemSfxEvent = "g10_sfx_event";          // market event card pop
    public const string StemSfxIpo = "g10_sfx_ipo";              // IPO lucky-draw pop
    public const string StemSfxDanmaku = "g10_sfx_danmaku";      // fund-holder danmaku wave
    public const string StemSfxAward = "g10_sfx_award";          // tier promotion fanfare
    public const string StemSfxGap = "g10_sfx_gap";              // overnight gap whoosh
    public const string StemSfxCandle = "g10_sfx_candle";        // daily K-line roll-up
    /// &lt;/summary&gt;The hook stamp owns t0 and starts the loop channels.&lt;/summary&gt;
    public const string StampTradeHook = "evt_trade_hook";
    /// &lt;/summary&gt;Stage stamps / ceremony beats -&gt; one-shot stems.&lt;/summary&gt;
    static readonly Dictionary&lt;string, string&gt; StampMap = new Dictionary&lt;string, string&gt;
    {
        { "evt_open_bell", StemSfxOpen },                        // the opening gong rides the market bell
        { "fill_sfx", StemSfxMatch },                            // the match chime trigger
        { "coin_sfx", StemSfxCoin },                              // the signature coin chime trigger
        { "evt_market_event_card", StemSfxEvent },              // the event card pop
        { "evt_rank_up", StemSfxAward },                          // the promotion ceremony
        { TradeTierGate.CeremonyAwardSfx, StemSfxAward },      // the ceremony sfx beat
    };
    /// &lt;/summary&gt;Named face cues (non-stamp faces) -&gt; one-shot stems.&lt;/summary&gt;
    public const string CueLimitSeal = "limit_seal";            // a held board seals the daily cap
    public const string CueTraderUpgrade = "trader_upgrade";    // the desk upgrades a trader
    public const string CueIpoDraw = "ipo_draw";                // the IPO lucky draw pops
    public const string CueDanmaku = "danmaku_wave";            // the fund-holder comment wave
    public const string CueOfflineGap = "offline_gap";          // the overnight gap surprise face
    public const string CueCloseSettle = "close_settle";        // the daily K-line roll-up
    static readonly Dictionary&lt;string, string&gt; CueMap = new Dictionary&lt;string, string&gt;
    {
        { CueLimitSeal, StemSfxLimit },
        { CueTraderUpgrade, StemSfxUpgrade },
        { CueIpoDraw, StemSfxIpo },
        { CueDanmaku, StemSfxDanmaku },
        { CueOfflineGap, StemSfxGap },
    }
}

```

```
        { CueCloseSettle, StemSfxCandle },
    };

    public static IReadOnlyDictionary<string, string> Stamps => StampMap;
    public static IReadOnlyDictionary<string, string> Cues => CueMap;
    /// <summary>The loop-channel stems (main BGM + hall bed).</summary>
    public static readonly string[] Channels = { StemBgmMain, StemAmbHall };
    /// <summary>Session-mark audio: the market's own ritual bells.</summary>
    public static string StemForSession(string sessionId)
    {
        if (string.IsNullOrEmpty(sessionId)) return null;
        if (sessionId == "open_bell") return StemSfxOpen;
        if (sessionId == "close_bell") return StemSfxClose;
        return null;
    }

    public static string StemForStamp(string stampId)
        => stampId != null && StampMap.TryGetValue(stampId, out var s) ? s : null;
    public static string StemForCue(string cueId)
        => cueId != null && CueMap.TryGetValue(cueId, out var s) ? s : null;
    // ---- playback (null-safe, headless-safe) ----
    static AudioSource _host;
    static readonly Dictionary<string, AudioSource> _loopChannels = new Dictionary<string, AudioSource>();
    static AudioSource Host()
    {
        if (_host == null)
        {
            var go = new GameObject("G10AudioHost");
            UnityEngine.Object.DontDestroyOnLoad(go);
            _host = go.AddComponent<AudioSource>();
        }
        return _host;
    }

    /// <summary>Fire-and-forget one-shot; false = missing/skipped, never throws.</summary>
    public static bool Play(string stem)
    {
        try
        {
            var clip = AudioRuntime.Get(stem);
            if (clip == null) return false;
            Host().PlayOneShot(clip);
            return true;
        }
        catch (Exception)
        {
            return false; // headless/batchmode: silence is the legal fallback
        }
    }

    public static bool PlayStamp(string stampId) => Play(StemForStamp(stampId));
    public static bool PlayCue(string cueId) => Play(StemForCue(cueId));
    public static bool PlaySession(string sessionId) => Play(StemForSession(sessionId));
    /// <summary>Start (or keep) a loop channel; false = missing/skipped.</summary>
```

```
public static bool StartChannel(string stem)
{
    try
    {
        var clip = AudioRuntime.Get(stem);
        if (clip == null) return false;
        if (!_loopChannels.TryGetValue(stem, out var src) || src == null)
        {
            var go = new GameObject("G10Audio_" + stem);
            UnityEngine.Object.DontDestroyOnLoad(go);
            src = go.AddComponent<AudioSource>();
            _loopChannels[stem] = src;
        }
        src.clip = clip;
        src.loop = true;
        if (!src.isPlaying) src.Play();
        return true;
    }
    catch (Exception)
    {
        return false;
    }
}

/// <summary>The main-face entry: the hook stamp owns t0, the main
/// BGM and the hall bed ride in with it (the channel law).</summary>
public static bool StartMainFace()
{
    bool bgm = StartChannel(StemBgmMain);
    bool amb = StartChannel(StemAmbHall);
    return bgm || amb;
}
}

// TradeBoard.cs
// TradeBoard.cs - the boards face (G10-T013, SYS06): the dragon list and
// the daily perf board - Board{kind,rows} under the revisit-refresh law.
// The dragon board ranks the day's top movers by |effective day move| and
// casts the hot-money crew (the roster's crew-domain seats, the
// actor-troupe law) as the row riders: a stable cameo pairing - the
// stock's own seed x the market day picks the seat, zero RNG, the same
// day always prints the same crew on the same movers. The player takes
// the rider seat on their own lived seals: a gang-up push that sealed
// (the SYS07 dragon feed) or a held stock sealing its board = the
// dragon-list appearance (the E04/E08 face), booked once per day-stock
// as evt_dragon_face. The perf board ranks the player's book against the
// crew: the player's day return marks the live book against the day's
// open print, the crew's day returns are pure seed-derivations inside
// their frozen bands (the phantom leaderboard: zero server, zero RNG,
// the same day prints the same crew books). Refresh() is the only
// writer: the host calls it on every visit (the boards never tick on
```

```

// their own - they catch up on the return), and a same-state refresh
// prints the same board twice. The showcase card carries the board as an
// optional data-source slot (CardLine - the day's perf rank plus the top
// mover). Pure C# zero-RNG: the same script always fingerprints
// byte-identical.
using System.Collections.Generic;
using System.Globalization;
using System.Text;
namespace Sy.CrazyStocks
{
    /// <summary>One board row (SYS06 schema: Board{kind,rows}).</summary>
    public sealed class BoardRow
    {
        public string kind = ""; // "dragon" | "perf"
        public int day; // the market day at the refresh
        public int rank; // 1-based inside the kind
        public string actorId = ""; // "you" | the frozen crew seat id
        public string nameKey = ""; // the signature copy key
        public string stockId = ""; // the mover (dragon rows only)
        public int moveBp; // dragon: the mover's day bp; perf: the book's day return
    }
    public sealed class TradeBoard
    {
        public const string KindDragon = "dragon";
        public const string KindPerf = "perf";
        public const string ActorYou = "you";
        public const int DragonRowsMax = 3; // the movers podium
        readonly TradeMarket _market;
        readonly TradeIpoFund _ipoFund;
        readonly List<BoardRow> _rows = new List<BoardRow>();
        readonly List<string> _funnel = new List<string>();
        readonly HashSet<string> _faced = new HashSet<string>();
        public TradeBoard(TradeMarket market, TradeIpoFund ipoFund)
        {
            _market = market;
            _ipoFund = ipoFund;
        }
        // ---- live faces ----
        public TradeMarket Market => _market;
        public TradeIpoFund IpoFund => _ipoFund;
        public IReadOnlyList<BoardRow> Rows => _rows;
        public IReadOnlyList<string> Funnel => _funnel;
        // ---- the player's day book (the perf face) ----
        /// <summary>The book marked at the day's open print: the wallet
        /// plus every position at its open (the day-return baseline, a
        /// pure derivation off the live state).</summary>
        public int OpenBookHc()
        {
            int sum = _market.WalletHc;
            foreach (var def in TradeData.Stocks())

```

```

    {
        var s = _market.Stock(def.id);
        if (s != null) sum += s.shares * s.openHc;
    }
    return sum;
}

/// <summary>The player's day return in bp: the live book against
/// the open-marked book (zero when the book is empty - a leek
/// with no positions swings nothing).</summary>
public int PlayerDayBp()
{
    int open = OpenBookHc();
    if (open <= 0) return 0;
    long delta = (long)_market.TotalAssetsHc - open;
    return (int)(delta * 10000 / open);
}

/// <summary>The cameo rider for a mover (the actor-troupe law):
/// the stock's own seed x the market day picks the crew seat - a
/// stable cameo, zero RNG (the same day always casts the same
/// seat on the same mover).</summary>
public static OpponentDef CameoFor(StockDef def, int day)
{
    var crew = TradeData.BoardOpponents();
    uint h = TradeData.MixPublic(def.seed, (uint)day);
    return crew[h % (uint)crew.Length];
}

/// <summary>The player's lived seal on a mover: their own gang-up
/// push that sealed today (the SYS07 dragon feed) or a held
/// stock sealing its board at the refresh = the dragon-list
/// appearance face.</summary>
bool PlayerSealed(StockState s, int day, int bp)
{
    if (_ipoFund != null)
        foreach (var d in _ipoFund.DragonList)
            if (d.day == day && d.stockId == s.def.id) return true;
    int limitBp = s.board.limitPct * 100;
    return s.shares > 0 && bp >= limitBp;
}

// ---- the boards (a pure derivation of the market state) ----
/// <summary>Computes both boards off the live market state (pure:
/// no telemetry, no storage - Refresh stores, CardLine reads
/// live). The dragon podium ranks by |day move| with the ordinal
/// id tie-break; the perf board ranks the player's book against
/// the crew by day return desc with the id tie-break.</summary>
List<BoardRow> ComputeRows()
{
    int day = _market.Day;
    var rows = new List<BoardRow>();
    var movers = new List<StockState>();
    foreach (var def in TradeData.Stocks())

```

```
{
    var s = _market.Stock(def.id);
    if (s != null) movers.Add(s);
}
movers.Sort((a, b) =>
{
    int ba = _market.EffectiveDayBp(a), bb = _market.EffectiveDayBp(b);
    int absA = ba < 0 ? -ba : ba, absB = bb < 0 ? -bb : bb;
    if (absA != absB) return absB - absA;    // the bigger |move| first
    return string.CompareOrdinal(a.def.id, b.def.id);
});
int dragonRank = 0;
foreach (var s in movers)
{
    if (dragonRank >= DragonRowsMax) break;
    int bp = _market.EffectiveDayBp(s);
    bool you = PlayerSealed(s, day, bp);
    var cameo = CameoFor(s.def, day);
    rows.Add(new BoardRow
    {
        kind = KindDragon,
        day = day,
        rank = ++dragonRank,
        actorId = you ? ActorYou : cameo.id,
        nameKey = you ? TradeTexts.KeyBoardYou : cameo.nameKey,
        stockId = s.def.id,
        moveBp = bp,
    });
}
var books = new List<(string id, string key, int bp)>
{
    (ActorYou, TradeTexts.KeyBoardYou, PlayerDayBp()),
};
foreach (var o in TradeData.BoardOpponents())
    books.Add((o.id, o.nameKey, TradeData.OpponentDayBp(o, day)));
books.Sort((x, y) =>
{
    if (x.bp != y.bp) return y.bp - x.bp;    // the fatter day book first
    return string.CompareOrdinal(x.id, y.id);
});
int perfRank = 0;
foreach (var b in books)
    rows.Add(new BoardRow
    {
        kind = KindPerf,
        day = day,
        rank = ++perfRank,
        actorId = b.id,
        nameKey = b.key,
        stockId = "",
    });
}
```

```

        moveBp = b.bp,
    });
    return rows;
}
/// <summary>The revisit-refresh law: the only writer. The host
/// calls this on every visit (board open, return from offline) -
/// the rows land on the market's current day and state, the
/// appearance telemetry books once per day-stock.</summary>
public void Refresh()
{
    var fresh = ComputeRows();
    _rows.Clear();
    _rows.AddRange(fresh);
    foreach (var r in _rows)
        if (r.kind == KindDragon && r.actorId == ActorYou) Face(r.stockId, r.day);
}
void Face(string stockId, int day)
{
    string key = day + ":" + stockId;
    if (_faced.Add(key)) _funnel.Add("evt_dragon_face@" + key);
}
/// <summary>The showcase card slot (the optional data-source
/// seat): the day's perf rank plus the top mover, a live pure
/// derivation (the card never waits for a refresh).</summary>
public string CardLine()
{
    var rows = ComputeRows();
    int rank = 1, total = 0;
    foreach (var r in rows)
    {
        if (r.kind != KindPerf) continue;
        total++;
        if (r.actorId == ActorYou) rank = r.rank;
    }
    string topMover = "", topPct = "";
    foreach (var r in rows)
        if (r.kind == KindDragon && r.rank == 1)
        {
            topMover = r.stockId;
            topPct = PctStr(r.moveBp);
            break;
        }
    return string.Format(CultureInfo.InvariantCulture,
        TradeTexts.EnText(TradeTexts.KeyboardCardLine), rank, total, topMover, topPct);
}
static string PctStr(int bp)
    => (bp &= 0 ? "+" : "") + (bp / 100.0).ToString("0.0", CultureInfo.InvariantCulture) + "%";
/// <summary>Canonical fingerprint - the zero-RNG parity
/// instrument (the stored rows plus the appearance ledger).</summary>
public string Fingerprint()

```

```

        {
            var sb = new StringBuilder();
            sb.Append("day").Append(_market.Day);
            sb.Append("|rows").Append(_rows.Count);
            foreach (var r in _rows)
                sb.Append(';').Append(r.kind).Append('/').Append(r.rank).Append('@').Append(r.actorId)
                    .Append('+').Append(r.stockId).Append('^').Append(r.moveBp);
            sb.Append("|face").Append(_faced.Count);
            foreach (var f in _funnel) sb.Append('#').Append(f);
            return sb.ToString();
        }
    }
}

// TradeContent.cs
// TradeContent.cs - the S1 content-debt trio (G10-T015): the sadness
// economy chain (M16), the intel subscription desk (M17) and the daily
// replay theater (M23) - three light mechanisms riding the existing faces
// (the halving law): the close-bell event hands them their daily beat
// exactly like the tier-ticket desk does, the copy rides the M11 text
// engine (compiled English source + zh data file), and every value
// derives zero-RNG from the frozen wave math (the same day always
// prints the same numbers).
// M16 sadness economy: the noodle bar / rooftop cafe / fireworks
// stand - three counter-cyclical assets whose daily yield moves
// against the market index (the whole city eats noodles when the
// tape bleeds), with an own-wave component so a hedge is never a
// risk-free arbitrage (a flat tape still wobbles the yield band).
// M17 intel subscription: the three guru schools from the meme album
// (square-dance / mystic / technical) each sell rumor-grade intel -
// accuracy parked around one half (the true-false-half law), a trust
// that lands on a false call fires the comedy scam event (never a
// game over), and the third scam trips the tricked-thrice
// achievement (once, forever).
// M23 replay theater: the LIVE close bell composes the day's
// operations into the channel news (the template pool rotates
// zero-RNG by day, the album cast rotates the guest commentary);
// a REPLAY session never books a formal replay (rehearsals stay
// free of ceremony) and a closed day never rings the bell at all.
using System;
using System.Collections.Generic;
using System.Text;
namespace Sy.CrazyStocks
{
    public sealed class TradeContent
    {
        // ---- v1 wiring (sweep-tunable; the doc pack owns the faces) ----
        public const int SadnessAssetCount = 3;
        public const int IntelSchoolCount = 3;
        public const int ReplayTemplateCount = 6;
        public const int ReplayCastCount = 9; // the meme-album trader desk rotates the guest seat
    }
}

```

```

public const int ScamAchievementAt = 3;          // the tricked-thrice degree
public const int SadNoodleBetaPer mille = 900;   // noodles track the sadness the tightest
public const int SadCafeBetaPer mille = 700;     // the wind-blowers are a softer hedge
public const int SadFireworksBetaPer mille = 600; // tears need a budget
public const int SadnessWaveBandBp = 120;       // the own-wave band: never a risk-free hedge
public const int IntelFeeDanceHc = 800;
public const int IntelFeeMysticHc = 1200;
public const int IntelFeeTechnicalHc = 1500;
public const int IntelAccuracyDancePer mille = 550; // rumor-grade: a touch over half
public const int IntelAccuracyMysticPer mille = 450; // the dream chart reads under half
public const int IntelAccuracyTechnicalPer mille = 500;
public const string SadNoodleId = "sad_noodle_bar";
public const string SadCafeId = "sad_rooftop_cafe";
public const string SadFireworksId = "sad_fireworks_stand";
public const string SchoolDanceId = "school_square_dance";
public const string SchoolMysticId = "school_mystic";
public const string SchoolTechnicalId = "school_technical";
/// &lt;summary&gt;A counter-cyclical sadness asset (M16): the beta is
/// the anti-correlation weight against the market index, the seed
/// drives the own-wave (the never-risk-free band).&lt;/summary&gt;
public sealed class SadAssetDef
{
    public string id = "";
    public string nameKey = "";
    public int betaPer mille;
    public int seed;
}
/// &lt;summary&gt;A guru school (M17): rumor-grade intel at a
/// subscription fee - accuracy parked around one half (the
/// true-false-half law, never the pure truth, never pure
/// fiction).&lt;/summary&gt;
public sealed class SchoolDef
{
    public string id = "";
    public string nameKey = "";
    public int feeHc;
    public int accuracyPer mille;
    public int seed;
}
static readonly SadAssetDef[] SadTable =
{
    new SadAssetDef { id = SadNoodleId, nameKey = TradeTexts.KeySadAssetNoodle, betaPer mille = SadNoodle
    new SadAssetDef { id = SadCafeId, nameKey = TradeTexts.KeySadAssetCafe, betaPer mille = SadCafeBeta
    new SadAssetDef { id = SadFireworksId, nameKey = TradeTexts.KeySadAssetFireworks, betaPer mille = SadFirework
};
static readonly SchoolDef[] SchoolTable =
{
    new SchoolDef { id = SchoolDanceId, nameKey = TradeTexts.KeyIntelSchoolDance, feeHc = IntelFeeDanceH
    new SchoolDef { id = SchoolMysticId, nameKey = TradeTexts.KeyIntelSchoolMystic, feeHc = IntelFeeMyst
    new SchoolDef { id = SchoolTechnicalId, nameKey = TradeTexts.KeyIntelSchoolTechnical, feeHc = IntelFeeTechni

```

```

};
readonly TradeMarket _market;
readonly List<string> _funnel = new List<string>();
readonly List<string> _replayLines = new List<string>();
readonly Dictionary<string, long> _positions = new Dictionary<string, long>(); // invested principa
int _scammed;
bool _degreeTripped;
public TradeContent(TradeMarket market)
{
    _market = market;
}
// ---- live faces ----
/// <summary>The session clock mode (the M15 face): the LIVE close
/// books the formal replay theater, a REPLAY session never books
/// one. The session View driver sets the face at the S2.5 wiring;
/// the stage default is the formal face.</summary>
public ClockMode Mode = ClockMode.Live;
public IReadOnlyList<string> Funnel => _funnel;
public IReadOnlyList<string> ReplayLines => _replayLines;
public int ScammedTimes => _scammed;
public bool DegreeTripped => _degreeTripped;
public static IReadOnlyList<SadAssetDef> SadAssets => SadTable;
public static IReadOnlyList<SchoolDef> Schools => SchoolTable;
public static SadAssetDef SadAsset(string id)
{
    foreach (var a in SadTable) if (a.id == id) return a;
    return null;
}
public static SchoolDef School(string id)
{
    foreach (var s in SchoolTable) if (s.id == id) return s;
    return null;
}
/// <summary>The market index print: the whole-board average day
/// bp (the whole city watches one tape). Zero-RNG - every input
/// is frozen wave math plus booked disturbances.</summary>
public int MarketIndexBp()
{
    var stocks = TradeData.Stocks();
    long sum = 0;
    int n = 0;
    foreach (var def in stocks)
    {
        var s = _market.Stock(def.id);
        if (s == null) continue;
        sum += _market.EffectiveDayBp(s);
        n++;
    }
    return n == 0 ? 0 : (int)(sum / n);
}

```

```
/// <summary>The sadness yield law (M16): the daily return moves
/// beta-times against the index (the noodle bar booms when the
/// tape bleeds), plus the own wave so a flat tape still wobbles -
/// never a risk-free hedge. Pure derivation, zero RNG.</summary>
public static int SadnessYieldBp(SadAssetDef asset, int indexBp, int day)
{
    int hedge = -(indexBp * asset.betaPer mille) / 1000;
    long mix = TradeData.MixPublic(asset.seed, (uint)day);
    int wave = (int)(mix % (2 * SadnessWaveBandBp + 1)) - SadnessWaveBandBp;
    return hedge + wave;
}

/// <summary>Invests gold hundredths into a sadness asset (the
/// counter-cyclical hedge position; the wallet face wires at the
/// S2.5 numeric sweep). Booked on the content funnel.</summary>
public bool Invest(string assetId, long goldHc)
{
    if (SadAsset(assetId) == null || goldHc <= 0) return false;
    _positions.TryGetValue(assetId, out long held);
    _positions[assetId] = held + goldHc;
    Book("sad_invest", assetId, goldHc.ToString());
    return true;
}

public long SadnessValueHc(string assetId)
{
    _positions.TryGetValue(assetId, out long v);
    return v;
}

/// <summary>The rumor-grade truth flag (M17): the school's own
/// mixed wire - deterministic per school-day, parked around one
/// half (the true-false-half law).</summary>
public static bool IntelCallIsTrue(SchoolDef school, int day)
    => (long)(TradeData.MixPublic(school.seed, (uint)day) % 1000) < school.accuracyPer mille;

/// <summary>The day's call: the target pick and the printed
/// direction. On a true call the pick matches the actual day
/// print; on a false call the wire got it backwards - that is
/// what a rumor is (comedy, never advice).</summary>
public (string stockId, bool saysUp, bool wasTrue) IntelCall(SchoolDef school, int day)
{
    var stocks = TradeData.Stocks();
    int pick = (int)(TradeData.MixPublic(school.seed + 13, (uint)day) % (uint)stocks.Length);
    var s = _market.Stock(stocks[pick].id);
    bool actualUp = s != null && _market.EffectiveDayBp(s) > 0;
    bool wasTrue = IntelCallIsTrue(school, day);
    bool saysUp = wasTrue ? actualUp : !actualUp;
    return (stocks[pick].id, saysUp, wasTrue);
}

/// <summary>Trusting the day's call: the subscription fee books,
/// a true call books the hit, a false call fires the comedy scam
/// event (never a game over) and the third scam trips the
/// tricked-thrice achievement (once, forever).</summary>
```

```

public bool TrustIntel(string schoolId, int day)
{
    var school = School(schoolId);
    if (school == null) return false;
    Book("fee_intel_sub", schoolId, school.feeHc.ToString());
    var call = IntelCall(school, day);
    if (call.wasTrue)
    {
        Book("evt_intel_trust", schoolId, "hit:" + call.stockId);
        return true;
    }
    Book("evt_intel_scam", schoolId, call.stockId); // the comedy event: the desk eats noodles, the run lives
    _scammed++;
    if (_scammed == ScamAchievementAt && !_degreeTripped)
    {
        _degreeTripped = true;
        Book("evt_achievement_tricked_thrice", schoolId, "degree");
    }
    return false;
}

/// &summary&The close-bell beat (wired by TradeStage exactly like
/// the tier-ticket desk): the sadness ledger settles the day's
/// counter-cyclical yields and the replay theater composes the
/// channel news. The LIVE close books the formal replay; a REPLAY
/// session never books one (rehearsals stay free of ceremony); a
/// closed day never rings this bell at all.&summary&
public void OnCloseBell(int day)
{
    int indexBp = MarketIndexBp();
    foreach (var a in SadTable)
    {
        int y = SadnessYieldBp(a, indexBp, day);
        _positions.TryGetValue(a.id, out long v);
        if (v > 0)
        {
            long grown = v + v * y / 10000;
            if (grown < 0) grown = 0; // a hedge wobbles, never below the floor
            _positions[a.id] = grown;
        }
        Book("evt_sadness_yield", a.id, "d" + day + ":" + y + ":index" + indexBp);
    }
    if (Mode != ClockMode.Live)
    {
        Book("evt_replay_theater", "day" + day, Mode == ClockMode.Replay ? "replay_skip" : "closed_skip");
        return;
    }
    string line = ComposeReplay(day, indexBp);
    _replayLines.Add(line);
    Book("evt_replay_theater", "day" + day, "live:" + ReplayTemplateKey(day));
}

```

```
/// <summary>The replay template pool key (zero-RNG rotation by
/// day): replay_tpl_1 .. replay_tpl_6.</summary>
public static string ReplayTemplateKey(int day)
    => "replay_tpl_" + (1 + day % ReplayTemplateCount);
/// <summary>The guest commentary seat rotates the meme-album
/// trader desk (the actor-troupe law, zero RNG by day).</summary>
public static string ReplayCastKey(int day)
{
    string[] cast =
    {
        TradeTexts.KeyTraderAllInBro, TradeTexts.KeyTraderSteadyWang, TradeTexts.KeyTraderCounterLi,
        TradeTexts.KeyTraderGrannyInfo, TradeTexts.KeyTraderBoardSnatcher, TradeTexts.KeyTraderTopEscapeSis,
        TradeTexts.KeyTraderLeekTrainer, TradeTexts.KeyTraderNeverWrongDing, TradeTexts.KeyTraderSweeperMonk,
    };
    return cast[day % ReplayCastCount];
}
/// <summary>Composes the channel news (M23): the day's operations
/// fill the rotating template slots and the album cast rotates
/// the guest commentary. Slots: {0} fills / {1} top mover id /
/// {2} top mover bp / {3} wallet gold / {4} the market index bp
/// / {5} the guest commentary line.</summary>
public string ComposeReplay(int day, int indexBp)
{
    string topId = "";
    int topBp = 0;
    foreach (var def in TradeData.Stocks())
    {
        var s = _market.Stock(def.id);
        if (s == null) continue;
        int bp = _market.EffectiveDayBp(s);
        if (string.IsNullOrEmpty(topId) || Math.Abs(bp) > Math.Abs(topBp))
        {
            topId = def.id;
            topBp = bp;
        }
    }
    string commentator = TradeTexts.EnText(ReplayCastKey(day));
    return string.Format(TradeTexts.EnText(ReplayTemplateKey(day)),
        _market.Fills.Count, topId, topBp, _market.WalletHc, indexBp, commentator);
}
/// <summary>Canonical fingerprint - the zero-RNG parity instrument
/// for the content stream (the same script over the same days is
/// always byte-identical).</summary>
public string Fingerprint()
{
    var sb = new StringBuilder();
    sb.Append("mode").Append(Mode)
        .Append("|scams").Append(_scammed)
        .Append("|degree").Append(_degreeTripped ? 1 : 0);
    foreach (var f in _funnel) sb.Append(';').Append(f);
}
```

```

        foreach (var kv in _positions) sb.Append("|pos:").Append(kv.Key).Append('=').Append(kv.Value);
        foreach (var l in _replayLines) sb.Append("|replay:").Append(l);
        return sb.ToString();
    }
    void Book(string evt, string id, string slice)
    {
        _funnel.Add(evt + "@" + id + ":" + slice);
    }
}

// TradeData.cs
// TradeData.cs - frozen data schemas and tables for Crazy Trade (G10-T004).
// The numeric pack doc (G10 v1, the S0 doc face) is the source of truth and
// this file is its compiled mirror: schema freeze means the structural laws
// live here as code and DataSchemaCheck proves every law on every row.
// Values are sweep-tunable later (bot sweeps re-freeze revisions through the
// doc pack), the structure only moves with a schema version bump. Zero
// server faces and a zero-IAP economy: the wave is a deterministic pure-C#
// derivation (same seed = same bytes, replay parity by construction), the
// fee table only recycles in-game gold, and no cash-for-gain path exists
// anywhere in the schema (red line 12 + the compliance face).
namespace Sy.CrazyStocks
{
    /// <summary>One tier gate row (M04, dual track): the meme asset threshold
    /// AND the flag event together - a tier is a new gameplay ticket, never a
    /// pure numeric wall.</summary>
    public sealed class TierDef
    {
        public int tier;           // 1-5 (T1 leek retail .. T5 fund manager)
        public long assetGate;     // the meme asset threshold (gold)
        public string unlockId = ""; // the gameplay ticket this tier unlocks
        public string flagEventId = ""; // the must-have-experienced flag event
    }
    /// <summary>One board row on the wave domain (M02): the daily price
    /// limit, the wave band, the event sensitivity and the tier gate.</summary>
    public sealed class BoardDef
    {
        public string id = "";
        public int limitPct;       // daily limit up/down (+/- percent)
        public int bandPct;       // daily wave band (percent)
        public int eventSensitivityPercent; // 100 = normal, 150 = event-sensitive
        public int minTier;       // tier gate (1 = open from the start)
    }
    /// <summary>One meme stock row (the wave derives from its seed).</summary>
    public sealed class StockDef
    {
        public string id = "";
        public string boardId = "";
        public int seed;          // the deterministic wave seed
    }
}

```

```

/// &summaryOne trading-day session row (M05): the compressed real
/// minutes, the yield factor, the ritual flags and the tail window.&/summary
public sealed class SessionDef
{
    public string id = "";
    public string clock = "";    // the wall-clock label (ceremony anchors)
    public int realMinutes;      // ritual points carry 0
    public int yieldPercent;     // 50/100/30/100 on the four yield windows
    public bool ritual;          // the two bell ceremonies
    public bool tailWindow;      // the afternoon tail-move window
    public int tailWindowPercent; // the tail multiplier (150 = 1.5x)
    public int tailMinTier;      // the tail window tier gate
}

/// &summaryOne trader card row (M03): a passive plus a comedy side
/// effect - every card personality, zero insider-information faces
/// (compliance law: the rumor mill is square-dance-grade public comedy).&/summary
public sealed class TraderDef
{
    public string id = "";
    public int unlockTier;       // the tier the card joins the desk
    public string passiveId = "";
    public string sideEffectId = "";
    public bool hidden;          // the easter-egg seat (the sweeping monk, M13)
}

/// &summaryOne hot-money crew seat (SYS06, the board cameo roster:
/// the roster's crew-domain seats under the actor-troupe law - the
/// name key carries the forum-signature one-liner, the seed derives
/// the seat's day book, the band is the comedy profile width).&/summary
public sealed class OpponentDef
{
    public string id = "";
    public int seed;             // the deterministic day-book seed
    public int bandBp;           // the day-return band in bp
    public string nameKey = "";  // the signature copy key (TradeTexts)
}

/// &summaryOne market-event card row (M06, schema v2): the comedy
/// market-event pool E01-E20 - the tier gate, the family, the title copy
/// key and the chain pointer. Effects stay sweep-tunable proposals in
/// the numeric pack (the doc pack column 5); only the structure freezes
/// here. Chain ids are design pointers (" = no follow-up).&/summary
public sealed class EventDef
{
    public string id = "";       // E01..E20
    public int minTier;          // the tier gate (doc pack column 2)
    public string family = "";   // market/social/comedy/milestone/offline/global/achievement/broadcast/ritual
    public string titleKey = ""; // the closed-set copy key (TradeTexts)
    public string chainId = "";  // the follow-up pointer (" = no chain)
}

/// &summaryOne drama-theater pool row (M21, the SYS04 tier-event
/// additive): a closed comedy event the joint-board face can raise.

```

```
/// Rows th01-th06 are the miss-face blind-box closed set, th07-th08
/// ride the seal/streak bonus faces. The families stay inside the
/// SYS04 closed family set (never a new family), the letter never
/// fines (TheaterLetterFinePct = 0) and every word is pure theater -
/// a fictional regulator NPC, zero real-technique wording (the
/// compliance law).&lt;/summary&gt;
public sealed class TheaterDef
{
    public string id = "";          // th01..th08
    public string family = "";      // rides the SYS04 closed family set
    public string titleKey = "";    // the closed-set copy key (TradeTexts)
}
/// &lt;/summary&gt;One fund-sentiment redemption-tide pool row (M22, the T5
/// holder-face additive): a closed comedy row the tide face can
/// raise while holders march out. The tide is comedy, never a death
/// spiral (the fund always rebuilds), the families stay inside the
/// SYS04 closed set, and every word is pure holder-culture farce -
/// zero real-technique wording (the compliance law).&lt;/summary&gt;
public sealed class FundTideDef
{
    public string id = "";          // rt01..rt06
    public string family = "";      // rides the SYS04 closed family set
    public string titleKey = "";    // the closed-set copy key (TradeTexts)
}
public static class TradeData
{
    public const int SchemaVersion = 3;
    public const int TierCount = 5;
    public const int BoardCount = 4;
    public const int SessionCount = 6;          // auction .. close bell
    public const int TraderCount = 9;
    public const int EventCount = 20;           // the market-event pool E01-E20
    public const int DayLengthMinutes = 30;     // one trading day = 30 real minutes
    public const int ReversionCycleDays = 5;     // the mean-reversion period in trading days
    public const int EventDisturbMinPct = 2;     // the event disturbance band 2-8% (closed set)
    public const int EventDisturbMaxPct = 8;
    // ---- zero-IAP compliance faces (the economy recycles gold only) ----
    public const bool IapEnabled = false;
    public const bool GachaEnabled = false;
    // ---- pacing anchors (the red line 11 assertion sources) ----
    public const int FirstCoinMaxMs = 1500;      // first booked coin &lt;= 1.5s
    public const int FirstFillMaxMs = 8000;      // first order fills &lt;= 8s
    public const int FirstMarketEventMaxMs = 10000; // first market event card &lt;= 10s
    public const int P1FirstTraderSeconds = 60;   // first trader on the desk + first fill
    public const int P50StrategySeconds = 300;    // the three-strategy desk takes shape
    public const int AdsPerDayMin = 3;
    // ---- the fee recycle valve (the gold economy loop, zero IAP) ----
    public const int TradeFeeBp = 50;            // 0.5% of the filled amount (the recycle valve)
    public const int IpoLuckPer mille = 3;       // 0.3% IPO allotment (T3 comedy: rarer than a lottery win)
    public const int IpoMinTier = 3;
```

```
public const int JointBoardCostPct = 2; // one joint board push costs 2% of holdings (T4)
public const int JointBoardMinTier = 4;
public const int JointBoardDailyCap = 3; // at most 3 pushes per trading day
public const int FundFeePerMillePerDay = 5; // 0.5% fund fee per game day (T5)
public const int FundMinTier = 5;
// M20 iceberg desk (T022a data layer, the numeric pack section 5 mirrors
// these five - the check reconciles code vs doc, both frozen here)
public const int IcebergMinTier = 3; // the iceberg desk rides the T3 big-player face
public const int IcebergBigLotShares = 10000; // a direct desk lot at/above this self-lifts
public const int IcebergSelfLiftPct = 3; // the self-lift premium a big direct lot pays
public const int IcebergBatchShares = 2500; // one iceberg batch stays under the big-lot line
public const int IcebergBatchesPerBell = 2; // iceberg batches landing per close bell
// M21 drama theater (T023a data layer, the numeric pack section 5
// mirrors these five - the reconciliation rides the IpoFundCheck
// theater face; additive on the frozen schema, zero structural move)
public const int TheaterMinTier = 4; // the theater rides the T4 joint-board face
public const int TheaterBlindBoxCount = 6; // the closed miss-outcome blind box set
public const int TheaterSnipeDisturbPct = 3; // the fictional hot-money counter-snipe band
public const int TheaterPoolCount = 8; // the tier-theater event pool (SYS04 additive)
public const int TheaterLetterFinePct = 0; // the comedy letter never fines (pure theater)
// M22 fund-sentiment three-line management (T024a data layer, the
// numeric pack section 5 mirrors these - the reconciliation rides
// the IpoFundCheck fund face; additive on the frozen schema, zero
// structural move; the tide is comedy, never a death spiral)
public const int FundNavWarnPct = 5; // NAV drawdown vs the issue anchor that starts pressuring sentim
public const int FundNavBreakPct = 12; // NAV drawdown that hard-breaks the sentiment line
public const int FundSentimentStart = 60; // the holder-mood gauge starts calm-ish (comedy)
public const int FundSentimentWarnLine = 40; // below = the forum flame war (the warn state)
public const int FundSentimentBreakLine = 20; // below = the redemption tide (the break state)
public const int FundTideRedeemPctPerBell = 10; // holders leaving per close bell while the tide runs
public const int FundLetterRecover = 10; // the reassurance letter restores 10 mood points
public const int FundRoadshowRecover = 20; // the livestream roadshow restores 20 mood points
public const int FundRoadshowTideHoldBells = 2; // the roadshow holds the tide for 2 close bells
public const int FundRedpacketAumPct = 1; // the gold red packet costs 1% of AUM and stops the tide instant
public const int FundRedpacketRecover = 15; // the red packet also lifts the mood gauge by 15
public const int FundTidePoolCount = 6; // the redemption-tide comedy pool closed set rt01-rt06
// ---- offline (SYS09, the overnight gap face) ----
public const int OfflineRatePercent = 50; // offline advances at half pace (series law)
public const int OfflineCapHours = 12; // per-absence cap
public const int GapMinPct = 0; // overnight gap band 0-5% (surprise/spook source)
public const int GapMaxPct = 5;
public const string OfflineDoubleCategory = "double"; // AdCap family category
public const int DoubleOfferPerSpan = 1; // one double offer per absence span
public const int DoubleWinMinPer200 = 1; // frequency window 1-5 per 200 (G08 T009 paradigm)
public const int DoubleWinMaxPer200 = 5;
// ---- second-run inheritance (the fund-manager ending) ----
public const int InheritGoldPercent = 20; // floor
public const bool InheritTraderCards = true; // the core desk follows along
public const bool InheritAlbumsKept = true; // the three albums + the curve history persist
public const bool InheritTierReset = true; // the spine restarts at the leek retail tier
```

```

// ---- tier gates (the meme numbers ride the row) ----
static readonly TierDef[] TierTable =
{
    new TierDef { tier = 1, assetGate = 10000L,          unlockId = "one_main_board_leek",      flagEventId = "
    new TierDef { tier = 2, assetGate = 500000L,        unlockId = "first_trader_auto_trade",  flagEventId = "e
    new TierDef { tier = 3, assetGate = 100000000L,     unlockId = "trader_trio_ipo_slippage", flagEventId = "e
    new TierDef { tier = 4, assetGate = 1000000000L,    unlockId = "joint_capital_dragon_list", flagEventId =
    new TierDef { tier = 5, assetGate = 100000000000L,  unlockId = "crazy_fund_manager_desk",  flagEventId = "
};
public static TierDef[] Tiers() => (TierDef[])TierTable.Clone();
// ---- the wave boards (the night board band rides the v1 proposal
// face - bot sweeps retune; the structure stays) ----
static readonly BoardDef[] BoardTable =
{
    new BoardDef { id = "board_main",  limitPct = 10, bandPct = 4,  eventSensitivityPercent = 100, minTier = 1 }
    new BoardDef { id = "board_gem",   limitPct = 10, bandPct = 6,  eventSensitivityPercent = 150, minTier = 1 }
    new BoardDef { id = "board_crazy", limitPct = 20, bandPct = 10, eventSensitivityPercent = 100, minTier = 3 }
    new BoardDef { id = "board_night", limitPct = 15, bandPct = 8,  eventSensitivityPercent = 100, minTier = 4 }
};
public static BoardDef[] Boards() => (BoardDef[])BoardTable.Clone();
public static BoardDef BoardFor(string boardId)
{
    foreach (var b in BoardTable) if (b.id == boardId) return b;
    return BoardTable[0];
}
// ---- the v1 meme stock set (one example row per pack board, two on
// the main board - the leek duo) ----
static readonly StockDef[] StockTable =
{
    new StockDef { id = "stk_leek_power",      boardId = "board_main",  seed = 20260921 },
    new StockDef { id = "stk_overtime_cola",   boardId = "board_main",  seed = 31415926 },
    new StockDef { id = "stk_limit_bio",       boardId = "board_gem",   seed = 27182818 },
    new StockDef { id = "stk_metaverse_pancake", boardId = "board_crazy", seed = 16180339 },
    new StockDef { id = "stk_square_dance_audio", boardId = "board_night", seed = 14142135 },
};
public static StockDef[] Stocks() => (StockDef[])StockTable.Clone();
/// <summary>Deterministic wave: pure integer mixing - zero RNG, the
/// same (seed, day) always derives the same basis-point move (replay
/// parity by construction). The noise rides the board band, folds
/// toward the mean every ReversionCycleDays (mean reversion) and
/// clamps inside the daily limit.</summary>
public static int WaveBp(long seed, int tradingDay, BoardDef board)
{
    uint h = Mix(seed, (uint)tradingDay);
    int spanBp = board.bandPct * 200 + 1;
    int noiseBp = (int)(h % (uint)spanBp) - board.bandPct * 100;    // -band..+band
    if (tradingDay > 0 && tradingDay % ReversionCycleDays == 0) noiseBp /= 2;    // the reversion pull
    int limitBp = board.limitPct * 100;
    if (noiseBp > limitBp) noiseBp = limitBp;
    if (noiseBp < -limitBp) noiseBp = -limitBp;
}

```

```

        return noiseBp;
    }
    static uint Mix(long seed, uint day)
    {
        // splitmix-style integer mixing - deterministic on every platform
        uint z = (uint)(seed & 0xFFFFFFFF) ^ 0x9E3779B9u ^ (day * 0x85EBCA6Bu);
        z = (z ^ (z >> 16)) * 0x21F0AAADu;
        z = (z ^ (z >> 15)) * 0x735A2D97u;
        return z ^ (z >> 15);
    }
    /// <summary>The shared integer-mixing primitive (public: the SYS06
    /// cameo pairing derives its stable seat pick off the same law).</summary>
    public static uint MixPublic(long seed, uint day) => Mix(seed, day);
    // ---- the trading-day clock (one game day = 30 real minutes; the
    // bells are ceremony points with no yield window) ----
    static readonly SessionDef[] SessionTable =
    {
        new SessionDef { id = "auction",    clock = "09:15", realMinutes = 1,  yieldPercent = 50,  ritual = false, t
        new SessionDef { id = "open_bell", clock = "09:30", realMinutes = 0,  yieldPercent = 0,   ritual = true,  ta
        new SessionDef { id = "morning",    clock = "09:30", realMinutes = 10, yieldPercent = 100, ritual = false, ta
        new SessionDef { id = "lunch",      clock = "11:30", realMinutes = 3,  yieldPercent = 30,  ritual = false, ta
        new SessionDef { id = "afternoon",  clock = "13:00", realMinutes = 12, yieldPercent = 100, ritual = false, ta
        new SessionDef { id = "close_bell", clock = "15:00", realMinutes = 0,  yieldPercent = 0,   ritual = true,  ta
    };
    public static SessionDef[] Sessions() => (SessionDef[])SessionTable.Clone();
    // ---- the trader desk (v2 unlock allocation: the first trader lands
    // at T2 = auto-trade begins, the desk holds exactly three at T3, the
    // rumor granny joins the hot-money tier, the limit-up squad member
    // rides the explicit T4 gate; the v2 seats join the fund-manager
    // endgame - never-wrong ding (8th, the pack leaves his tier open,
    // proposal: T5) and the hidden sweeping monk (9th, the explicit
    // T5 easter egg of M13); bot sweeps retune) ----
    static readonly TraderDef[] TraderTable =
    {
        new TraderDef { id = "trd_all_in_bro",    unlockTier = 2, passiveId = "passive_all_in_yield_x180",    sideE
        new TraderDef { id = "trd_steady_wang",    unlockTier = 3, passiveId = "passive_volatility_half",        sid
        new TraderDef { id = "trd_counter_li",      unlockTier = 3, passiveId = "passive_reverse_indicator",      sid
        new TraderDef { id = "trd_granny_info",     unlockTier = 4, passiveId = "passive_event_alert_x130",      side
        new TraderDef { id = "trd_board_snatcher",  unlockTier = 4, passiveId = "passive_limit_up_gain_x120",    side
        new TraderDef { id = "trd_top_escape_sis",  unlockTier = 5, passiveId = "passive_take_profit_15",        side
        new TraderDef { id = "trd_leek_trainer",    unlockTier = 5, passiveId = "passive_loss_refund_10",        sid
        new TraderDef { id = "trd_never_wrong_ding", unlockTier = 5, passiveId = "passive_never_wrong",        sid
        new TraderDef { id = "trd_sweeper_monk",    unlockTier = 5, passiveId = "passive_bottom_sweep_x108",    sid
    };
    public static TraderDef[] Traders() => (TraderDef[])TraderTable.Clone();
    // ---- the market-event pool (M06, schema v2): twenty comedy cards
    // E01-E20, the doc pack tables frozen row by row (tier gate, family,
    // title key, chain pointer); effects ride the numeric pack as
    // sweep-tunable proposals ----
    static readonly EventDef[] EventTable =

```

```
{
    new EventDef { id = "E01", minTier = 1, family = "market",      titleKey = "event_title_e01", chainId = "ch
    new EventDef { id = "E02", minTier = 1, family = "market",      titleKey = "event_title_e02", chainId = "ch
    new EventDef { id = "E03", minTier = 1, family = "market",      titleKey = "event_title_e03", chainId = ""
    new EventDef { id = "E04", minTier = 2, family = "social",      titleKey = "event_title_e04", chainId = "ch
    new EventDef { id = "E05", minTier = 2, family = "comedy",      titleKey = "event_title_e05", chainId = "ch
    new EventDef { id = "E06", minTier = 3, family = "milestone",   titleKey = "event_title_e06", chainId = "ch
    new EventDef { id = "E07", minTier = 3, family = "milestone",   titleKey = "event_title_e07", chainId = ""
    new EventDef { id = "E08", minTier = 4, family = "market",      titleKey = "event_title_e08", chainId = "ch
    new EventDef { id = "E09", minTier = 5, family = "comedy",      titleKey = "event_title_e09", chainId = ""
    new EventDef { id = "E10", minTier = 1, family = "offline",     titleKey = "event_title_e10", chainId = ""
    new EventDef { id = "E11", minTier = 1, family = "global",      titleKey = "event_title_e11", chainId = "ch
    new EventDef { id = "E12", minTier = 1, family = "comedy",      titleKey = "event_title_e12", chainId = "ch
    new EventDef { id = "E13", minTier = 2, family = "achievement", titleKey = "event_title_e13", chainId = ""
    new EventDef { id = "E14", minTier = 2, family = "achievement", titleKey = "event_title_e14", chainId = ""
    new EventDef { id = "E15", minTier = 3, family = "comedy",      titleKey = "event_title_e15", chainId = "ch
    new EventDef { id = "E16", minTier = 2, family = "social",      titleKey = "event_title_e16", chainId = "ch
    new EventDef { id = "E17", minTier = 1, family = "market",      titleKey = "event_title_e17", chainId = "ch
    new EventDef { id = "E18", minTier = 3, family = "comedy",      titleKey = "event_title_e18", chainId = "ch
    new EventDef { id = "E19", minTier = 2, family = "broadcast",   titleKey = "event_title_e19", chainId = "ch
    new EventDef { id = "E20", minTier = 1, family = "ritual",      titleKey = "event_title_e20", chainId = "c
};
public static EventDef[] Events() => (EventDef[])EventTable.Clone();
// ---- the drama-theater pool (M21, the SYS04 tier-event additive):
// eight frozen comedy rows - th01-th06 form the miss-face blind-box
// closed set (the fictional regulator letter, the hot-money snipe,
// the melon crowd, the accidental fame, the forum flames, the
// dance-troupe cascade), th07-th08 ride the bonus faces (the noodle
// special on the miss streak, the channel special on the seal).
// Pure theater, zero real-technique wording (the compliance law) ----
static readonly TheaterDef[] TheaterPool =
{
    new TheaterDef { id = "th01", family = "comedy",    titleKey = "theater_title_th01" },
    new TheaterDef { id = "th02", family = "market",   titleKey = "theater_title_th02" },
    new TheaterDef { id = "th03", family = "social",   titleKey = "theater_title_th03" },
    new TheaterDef { id = "th04", family = "comedy",   titleKey = "theater_title_th04" },
    new TheaterDef { id = "th05", family = "social",   titleKey = "theater_title_th05" },
    new TheaterDef { id = "th06", family = "comedy",   titleKey = "theater_title_th06" },
    new TheaterDef { id = "th07", family = "comedy",   titleKey = "theater_title_th07" },
    new TheaterDef { id = "th08", family = "broadcast", titleKey = "theater_title_th08" },
};
public static TheaterDef[] TheaterEvents() => (TheaterDef[])TheaterPool.Clone();
// ---- the fund-sentiment redemption-tide pool (M22, the T5
// holder-face additive): six frozen comedy rows - the farewell
// banner exit, the redemption express thread, the bank queue to
// the noodle bar, the strategic-withdrawal spin, the channel
// "where did the holders go" special, the dance-troupe exit
// parade. The tide is comedy, never a death spiral (the fund
// always rebuilds), the families stay inside the SYS04 closed
// set, zero real-technique wording (the compliance law) ----
```

```

static readonly FundTideDef[] FundTidePool =
{
    new FundTideDef { id = "rt01", family = "comedy",    titleKey = "fund_tide_title_rt01" },
    new FundTideDef { id = "rt02", family = "social",    titleKey = "fund_tide_title_rt02" },
    new FundTideDef { id = "rt03", family = "comedy",    titleKey = "fund_tide_title_rt03" },
    new FundTideDef { id = "rt04", family = "social",    titleKey = "fund_tide_title_rt04" },
    new FundTideDef { id = "rt05", family = "broadcast", titleKey = "fund_tide_title_rt05" },
    new FundTideDef { id = "rt06", family = "comedy",    titleKey = "fund_tide_title_rt06" },
};

public static FundTideDef[] FundTideEvents() => (FundTideDef[])FundTidePool.Clone();
// ---- the board cameo crew (SYS06): the frozen hot-money seats
// that ride the dragon rows and fill the perf board - every seat
// is a roster crew-domain cameo (the actor-troupe law), the day
// books are pure seed-derivations (the phantom leaderboard: zero
// server, zero RNG, the same day prints the same crew books) ----
public const int BoardOpponentCount = 4;
static readonly OpponentDef[] BoardOpponentTable =
{
    new OpponentDef { id = "opp_squad_admiral",    seed = 8848, bandBp = 2000, nameKey = "opp_squad_admiral" },
    new OpponentDef { id = "opp_floor_sky",        seed = 1314, bandBp = 1800, nameKey = "opp_floor_sky" },
    new OpponentDef { id = "opp_auction_bro",      seed = 915,  bandBp = 1200, nameKey = "opp_auction_bro" },
    new OpponentDef { id = "opp_one_day_tourist",   seed = 250,  bandBp = 1500, nameKey = "opp_one_day_tourist" },
};

public static OpponentDef[] BoardOpponents() => (OpponentDef[])BoardOpponentTable.Clone();
/// <summary>The crew seat's day return in bp (the phantom face):
/// the seat's own seed x the market day mixes inside its band -
/// deterministic, zero RNG (the same day always prints the same
/// book for the same seat).</summary>
public static int OpponentDayBp(OpponentDef o, int day)
{
    uint h = Mix(o.seed, (uint)day);
    int span = o.bandBp * 2 + 1;
    return (int)(h % (uint)span) - o.bandBp;
}

/// <summary>Canonical signature - the dual-derive byte-parity
/// instrument (the schema freeze proof): two derives of the frozen
/// tables agree here or nowhere.</summary>
public static string Signature()
{
    var sb = new System.Text.StringBuilder();
    sb.Append("v").Append(SchemaVersion);
    sb.Append("|tiers:");
    foreach (var t in TierTable)
        sb.Append(t.tier).Append('@').Append(t.assetGate).Append('/')
            .Append(t.unlockId).Append(',').Append(t.flagEventId).Append(';');
    sb.Append("|boards:");
    foreach (var b in BoardTable)
        sb.Append(b.id).Append('/').Append(b.limitPct).Append(',').Append(b.bandPct).Append(',')
            .Append(b.eventSensitivityPercent).Append(',').Append(b.minTier).Append(';');
    sb.Append("|stocks:");

```

```

foreach (var s in StockTable) sb.Append(s.id).Append('@').Append(s.boardId).Append('+').Append(s.seed).Append(s.wave).Append(ReversionCycleDays).Append(',').Append(EventDisturbMinPct).Append('-').Append(EventDisturbMaxPct)
foreach (var s in StockTable)
{
    var b = BoardFor(s.boardId);
    for (int day = 1; day <= ReversionCycleDays; day++)
        sb.Append(',').Append(WaveBp(s.seed, day, b));
}
sb.Append("|day:").Append(DayLengthMinutes);
foreach (var x in SessionTable)
    sb.Append(';').Append(x.id).Append('@').Append(x.clock).Append('/')
        .Append(x.realMinutes).Append(',').Append(x.yieldPercent).Append(',')
        .Append(x.ritual ? "R" : "-").Append(',').Append(x.tailWindow).Append(',').Append(x.tailWindowPercent)
sb.Append("|desk:");
foreach (var t in TraderTable)
    sb.Append(t.id).Append('/').Append(t.unlockTier).Append('/').Append(t.passiveId).Append('+').Append(t.isHidden ? "/H" : "").Append(';');
sb.Append("|events:");
foreach (var e in EventTable)
    sb.Append(e.id).Append('@').Append(e.minTier).Append('/').Append(e.family).Append('/')
        .Append(e.titleKey).Append('+').Append(string.IsNullOrEmpty(e.chainId) ? "-" : e.chainId).Append(';');
sb.Append("|fees:").Append(TradeFeeBp).Append(',').Append(IpoLuckPer mille).Append('@').Append(IpoMinTier)
    .Append(',').Append(JointBoardCostPct).Append('@').Append(JointBoardMinTier).Append('x').Append(JointBoard)
    .Append(',').Append(FundFeePer millePerDay).Append('@').Append(FundMinTier).Append(",iap=").Append(IapEnabl
sb.Append("|pacing:").Append(FirstCoinMaxMs).Append(',').Append(FirstFillMaxMs).Append(',').Append(FirstMark
    .Append(',').Append(P1FirstTraderSeconds).Append(',').Append(P50StrategySeconds).Append(',').Append(AdsPer
sb.Append("|offline:").Append(OfflineRatePercent).Append(',').Append(OfflineCapHours).Append(',')
    .Append(GapMinPct).Append('-').Append(GapMaxPct).Append(',').Append(OfflineDoubleCategory).Append(',')
    .Append(DoubleOfferPerSpan).Append(',').Append(DoubleWinMinPer200).Append(',').Append(DoubleWinMaxPer200);
sb.Append("|inherit:").Append(InheritGoldPercent).Append(',').Append(InheritTraderCards).Append(',')
    .Append(InheritAlbumsKept).Append(',').Append(InheritTierReset);
sb.Append("|boardOpp:");
foreach (var o in BoardOpponentTable)
    sb.Append(o.id).Append('@').Append(o.seed).Append('~').Append(o.bandBp).Append('+').Append(o.nameKey).Append(';')
return sb.ToString();
}
}
}
// TradeDesk.cs
// TradeDesk.cs - the weird-trader hiring engine (G10-T006, SYS03).
// The desk consumes the frozen TradeData trader table (T004): every card
// carries a passive on a named desk domain plus a comedy side effect - zero
// insider-information faces anywhere (the rumor mill stays square-dance-grade
// public comedy, the compliance law). The ten-card hiring spree is ad-funded
// (zero IAP, zero real gacha) and rides the G08 T009 three-piece law on the
// draw category: HasCategoryBudget precheck and ShowRewarded consumption
// share one policy instance and one day bucket, the evt_ad funnel books
// offer -& show -& complete -& landed in order, and a capped tap refuses
// BEFORE the ad ever plays. Seats are tier-capped (the T1 seat is the
// opening-gift chair the tutorial beat hires all-in bro into) and the desk

```

```

// auto-fills in table order - only seated traders pay their passive. The
// steady-wang passive wires the live market face (the intraday wiggle folds
// to x0.5 through TradeMarket.WiggleAmpBp); the remaining domains land on
// the desk profile the auto-trade and event faces consume as they unlock
// (T2 first_trader_auto_trade onward). Draw order is a deterministic
// rotation (zero RNG). Pure C#, no MonoBehaviour.
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Platform;
namespace Sy.CrazyStocks
{
    /// <summary>The seated-desk profile every passive folds into (identity
    /// defaults keep every legacy number untouched).</summary>
    public sealed class DeskProfile
    {
        public int StakePer mille = 900; // the desk auto-trade stake of the wallet (all-in bro: 1000)
        public int YieldVarBp = 10000; // his yield variance face x1.8 (the comedy risk dial)
        public int WiggleAmpBp = 10000; // steady wang: the live market wiggle x0.5
        public bool HalfPosition; // steady wang: forever half position
        public bool ReverseBetting; // counter li: the bet-against-him play unlocks
        public int EventAlertBp = 10000; // granny: the market-event early warning x1.3
        public int LimitUpGainBp = 10000; // board snatcher: limit-up gains x1.2
        public int TakeProfitPct; // top-escape sis: the +15% take-profit trigger
        public int SellFlightPct; // her sold-too-early comedy rate
        public int LossRefundPct; // leek trainer: losses refund 10% as "experience"
        public int MarketMadLectureBp = 10000; // never-wrong ding (v2): his "the market is mad" lecture rides the yield
        public int BottomSweepBp = 10000; // the hidden sweeping monk (v2): bottom-range buys sweep x1.08
        public bool IsIdentity
            => StakePer mille == 900 && YieldVarBp == 10000 && WiggleAmpBp == 10000 && !HalfPo
                && !ReverseBetting && EventAlertBp == 10000 && LimitUpGainBp == 10000
                && TakeProfitPct == 0 && SellFlightPct == 0 && LossRefundPct == 0
                && MarketMadLectureBp == 10000 && BottomSweepBp == 10000;
    }

    /// <summary>SYS03: the hiring desk. The scene host wires the ten-draw
    /// CTA to TryDrawTen and the desk to the market via ApplyTo; headless
    /// gates drive both directly.</summary>
    public sealed class TradeDesk
    {
        /// <summary>Draw point id (T006): draws from the draw 3/day category
        /// (AdCategories.CategoryOfPoint authority, g10_ prefix keeps id
        /// namespaces isolated).</summary>
        public const string DrawPoint = "g10_card_draw";
        public const int CardsPerDraw = 10;
        /// <summary>The opening-gift chair: the tutorial beat hires all-in
        /// bro at T1 (the <=10s comedy beat) - the only seat T1 holds.</summary>
        public const string OpeningGiftTraderId = "trd_all_in_bro";
        /// <summary>The promotion-spine hook (T008): the first hire IS the
        /// T2 lived experience - the desk raises the first-trader flag on
        /// the gate the moment a card lands on the payroll.</summary>

```

```

// seeds the wallet, the inherited desk seats and re-lives the first-trader
// experience, the run counter rolls, the ending re-arms. Pure C#
// zero-RNG: the same script always fingerprints byte-identical.
using System.Collections.Generic;
using System.Globalization;
using Sy.Social;
namespace Sy.CrazyStocks
{
    /// <summary>The frozen carry plan for the next run (the inheritance
    /// law compiled into data): the 20% floored gold, the kept trader
    /// ids and the run index the carry seeds.</summary>
    public sealed class NewRunCarry
    {
        public int GoldHc;
        public List<string> TraderIds = new List<string>();
        public int RunIndex;
    }
    public sealed class TradeShowcase
    {
        // ---- v1 wiring (sweep-tunable; the frozen laws live in TradeData) ----
        public const string CardTemplateStem = "g10_curve_card"; // art lands S2.5; null-safe procedural until then
        public const int CurveCellsMax = 14; // the run card: the last 14 closed days
        public const int EndingCellsMax = 30; // the ending long card: the whole run journey
        public const int CrazyFlatBandBp = 300; // |net| under 300bp = the city naps
        public const int CrazyGreedBandBp = 300; // net over +300bp = greed
        public const int CrazyFrenzyBandBp = 800; // net over +800bp = frenzy
        public const int CrazyPanicBandBp = 300; // net under -300bp = panic
        public const int CrazyIndexDivisorBp = 50; // |net|/50 clamped to 100
        TradeStage _stage;
        TradeDesk _desk;
        readonly List<string> _funnel = new List<string>();
        readonly HashSet<string> _traderAlbum = new HashSet<string>();
        readonly HashSet<string> _stockAlbum = new HashSet<string>();
        readonly HashSet<string> _eventAlbum = new HashSet<string>();
        readonly List<(int run, int day, int assetsHc)> _curve = new List<(int, int, int)>();
        int _runCount = 1;
        int _closeSeen; // the session-mark scan pointer (the curve sampler)
        int _fillSeen; // the fill scan pointer (the stock album)
        int _eventSeen; // the fire-log scan pointer (the event album)
        int _promoSeen; // the promotion scan pointer (the ending watch)
        bool _endingReached;
        public TradeShowcase(TradeStage stage, TradeDesk desk = null)
        {
            _stage = stage;
            _desk = desk;
        }
        /// <summary>The boards face (SYS06): the optional card
        /// data-source slot - null (the default) keeps the T011 four-line
        /// card spec untouched, an attached board rides the fifth line
        /// (the day's perf rank + the top mover).</summary>

```

```

public TradeBoard Board { get; set; }
// ---- live faces ----
public TradeStage Stage => _stage;
public TradeDesk Desk => _desk;
public int RunCount => _runCount;
public bool EndingReached => _endingReached;
public IReadOnlyList<string> Funnel => _funnel;
public IReadOnlyCollection<string> TraderAlbum => _traderAlbum;
public IReadOnlyCollection<string> StockAlbum => _stockAlbum;
public IReadOnlyCollection<string> EventAlbum => _eventAlbum;
public IReadOnlyList<(int run, int day, int assets)> Curve => _curve;
public int AlbumTotalMet => _traderAlbum.Count + _stockAlbum.Count + _eventAlbum.Count;
public int AlbumTotalPool => TradeData.TraderCount + TradeData.Stocks().Length + TradeData.EventCount;
/// <summary>Scans the owning systems since the last call: every
/// new close bell samples the curve at the bell print, every new
/// fill meets its meme stock, every new event fire meets its card,
/// every desk collection row meets its trader, and the promotion
/// spine reaching the fund-manager tier flips the ending. The
/// host calls this on its own tick cadence; idempotent per
/// increment (the scan pointers never double-count).</summary>
public void Observe()
{
    var marks = _stage.Market.SessionMarks;
    while (_closeSeen < marks.Count)
    {
        var m = marks[_closeSeen++];
        if (m.sessionId == "close_bell")
        {
            int assets = _stage.Market.TotalAssetsHc;
            _curve.Add((_runCount, m.day, assets));
            _funnel.Add("evt_curve_bell@" + _runCount + ":" + m.day + ":" + assets);
        }
    }
    var fills = _stage.Market.Fills;
    while (_fillSeen < fills.Count)
        _stockAlbum.Add(fills[_fillSeen++].stockId);
    var fires = _stage.Events.FireLog;
    while (_eventSeen < fires.Count)
        if (TryParseEventId(fires[_eventSeen++].line, out var eid))
            _eventAlbum.Add(eid);
    if (_desk != null)
        foreach (var t in _desk.Collection)
            _traderAlbum.Add(t.id);
    var proms = _stage.Gate.Promotions;
    while (_promoSeen < proms.Count)
    {
        var p = proms[_promoSeen++];
        if (p.tier == TradeData.TierCount && !_endingReached)
        {
            _endingReached = true;

```

```

        _funnel.Add("evt_fund_ending@run" + _runCount);
    }
}

}

/// <summary>Scans any owner's telemetry lines for event meetings
/// (the offline E10 gap card owns its funnel: one scan after a
/// settle meets the card).</summary>
public void ObserveFunnel(IEnumerable<string> lines)
{
    if (lines == null) return;
    foreach (var l in lines)
        if (TryParseEventId(l, out var eid))
            _eventAlbum.Add(eid);
}

/// <summary>Explicit event meeting (the offline summary face
/// wiring point). Idempotent per id.</summary>
public void UnlockEvent(string eid)
{
    if (!string.IsNullOrEmpty(eid)) _eventAlbum.Add(eid);
}

/// <summary>evt_market_event@&lt;eid&gt;... parser (the funnel
/// line head law every event owner follows).</summary>
public static bool TryParseEventId(string line, out string eid)
{
    eid = null;
    if (string.IsNullOrEmpty(line) || !line.StartsWith("evt_market_event@")) return false;
    int from = "evt_market_event@".Length;
    int to = line.IndexOf(':', from);
    if (to < 0) to = line.Length;
    eid = line.Substring(from, to - from);
    return eid.Length > 0;
}

// ---- the today crazy index (M25: the whole city's mood gauge) ----
/// <summary>The net effective day move over every listed meme
/// stock (the wave plus the live event disturbances, clamped per
/// board) - the mood input, pure derivation.</summary>
public static int NetMoveBp(TradeMarket market)
{
    int net = 0;
    foreach (var def in TradeData.Stocks())
    {
        var s = market.Stock(def.id);
        if (s != null) net += market.EffectiveDayBp(s);
    }
    return net;
}

/// <summary>The four-band closed set: frenzy over +800, greed
/// over +300, panic under -300, laid-back between.</summary>
public static string CrazyBandKeyOf(int netBp)
{

```

```

        if (netBp >= CrazyFrenzyBandBp) return TradeTexts.KeyCrazyFrenzy;
        if (netBp >= CrazyGreedBandBp) return TradeTexts.KeyCrazyGreed;
        if (netBp <= -CrazyPanicBandBp) return TradeTexts.KeyCrazyPanic;
        return TradeTexts.KeyCrazyFlat;
    }
    /// <summary>The 0-100 gauge face: |net|/50 clamped (5000bp of
    /// aggregate city movement reads the full house).</summary>
    public static int CrazyIndexPctOf(int netBp)
    {
        int a = netBp < 0 ? -netBp : netBp;
        int pct = a / CrazyIndexDivisorBp;
        return pct > 100 ? 100 : pct;
    }
    /// <summary>The live triple (net, gauge, band) off the market.</summary>
    public (int netBp, int pct, string bandKey) CrazyIndex()
    {
        int net = NetMoveBp(_stage.Market);
        return (net, CrazyIndexPctOf(net), CrazyBandKeyOf(net));
    }
    // ---- the curve cells (the formation layer of every card) ----
    /// <summary>Pure level bucket: 1-5 over the series min-max
    /// (a flat series reads the mid bucket 3).</summary>
    public static int CurveCellTier(int assetsHc, int minHc, int maxHc)
    {
        if (maxHc <= minHc) return 3;
        int t = 1 + (assetsHc - minHc) * 4 / (maxHc - minHc);
        if (t < 1) t = 1;
        if (t > 5) t = 5;
        return t;
    }
    /// <summary>The formation cells for an asset series tail (the
    /// last maxCells points): each closed day one cell, the bucket
    /// tier paints the level, stack 1 (the day count rides the copy
    /// lines, the cells paint the shape).</summary>
    public static CardCell[] CellsFor(List<int> assetsSeries, int maxCells)
    {
        var cells = new List<CardCell>();
        if (assetsSeries == null || assetsSeries.Count == 0) return cells.ToArray();
        int min = assetsSeries[0], max = assetsSeries[0];
        foreach (var a in assetsSeries)
        {
            if (a < min) min = a;
            if (a > max) max = a;
        }
        int from = assetsSeries.Count > maxCells ? assetsSeries.Count - maxCells : 0;
        for (int i = from; i < assetsSeries.Count; i++)
            cells.Add(new CardCell { Tier = CurveCellTier(assetsSeries[i], min, max), Stack = 1 });
        return cells.ToArray();
    }
    /// <summary>This run's curve points (the card face: the journey

```

```
/// belongs to the run, the history stays for the next one).</summary>
public List<int> RunCurveAssets()
{
    var list = new List<int>();
    foreach (var p in _curve)
        if (p.run == _runCount) list.Add(p.assetsHc);
    return list;
}
// ---- the three-in-one card (SYS08: curve + tier badge + crazy index) ----
// <summary>The run card spec: AutoCardComposer reuse - the title
// band, the assets/tier/crazy/collection copy lines, the share
// challenge footer, the kind/tier query payload and the curve
// formation layer (last 14 closed days).</summary>
public ShareCard ComposeCard()
{
    var m = _stage.Market;
    var crazy = CrazyIndex();
    var lines = new List<string>();
    {
        TradeTexts.EnText(TradeTexts.KeyStatAssets) + ": " + HcStr(m.TotalAssetsHc),
        TradeTexts.EnText(TradeTexts.KeyStatTier) + ": " + TradeTexts.EnText(TradeTierGate.TierTitleKey(_stage.G
        TradeTexts.EnText(crazy.bandKey) + " " + crazy.pct + "/100",
        string.Format(TradeTexts.EnText(TradeTexts.KeyDailyReport), m.Day,
            TradeTexts.EnText(TradeTexts.KeyAlbumTitle) + " " + AlbumTotalMet + "/" + AlbumTotalPool),
    };
    if (Board != null) lines.Add(Board.CardLine()); // the optional SYS06 slot (null = the untouched spec)
    var formation = CellsFor(RunCurveAssets(), CurveCellsMax);
    var card = AutoCardComposer.Card(CardTemplateStem,
        TradeTexts.EnText(TradeTexts.KeyCurveTitle),
        lines.ToArray(),
        TradeTexts.EnText(TradeTexts.KeyShareChallenge),
        "g10&kind=curve&tier=" + _stage.Gate.Tier + "&run=" + _runCount);
    card.Formation = formation;
    card.FormationCols = formation.Length;
    card.FormationRows = 1;
    return card;
}
// <summary>The ending long card spec: the fund-manager summit
// title over the whole-run curve (the long image face). Null
// until the promotion spine reaches the fund-manager tier.</summary>
public ShareCard ComposeEndingCard()
{
    if (!_endingReached) return null;
    var m = _stage.Market;
    var crazy = CrazyIndex();
    var lines = new List<string>();
    {
        TradeTexts.EnText(TradeTexts.KeyStatAssets) + ": " + HcStr(m.TotalAssetsHc),
        TradeTexts.EnText(TradeTexts.KeyStatTier) + ": " + TradeTexts.EnText(TradeTierGate.TierTitleKey(_stage.G
        TradeTexts.EnText(crazy.bandKey) + " " + crazy.pct + "/100",
```

```
        string.Format(TradeTexts.EnText(TradeTexts.KeyDailyReport), m.Day,
            TradeTexts.EnText(TradeTexts.KeyAlbumTitle) + " " + AlbumTotalMet + "/" + AlbumTotalPool),
    };
    if (Board != null) lines.Add(Board.CardLine()); // the optional SYS06 slot (null = the untouched spec)
    var formation = CellsFor(RunCurveAssets(), EndingCellsMax);
    var card = AutoCardComposer.Card(CardTemplateStem,
        TradeTexts.EnText(TradeTexts.KeyEndingTitle),
        lines.ToArray(),
        TradeTexts.EnText(TradeTexts.KeyShareChallenge),
        "g10&kind=ending&tier=" + _stage.Gate.Tier + "&run=" + _runCount);
    card.Formation = formation;
    card.FormationCols = formation.Length;
    card.FormationRows = 1;
    return card;
}

/// <summary>Composes and saves one card through the shared
/// ShareCardService: a successful compose books the evt_curve_share
/// telemetry line (kind/tier slices, the north-star share-rate
/// source); a failed compose books nothing.</summary>
public ComposeResult ComposeAndSave(ShareCardService svc, ShareCard card, string kind)
{
    var res = svc.Compose(card);
    if (res != null & res.Ok)
        _funnel.Add("evt_curve_share@" + kind + ":" + _stage.Gate.Tier);
    return res;
}

// ---- the ending and the second-run inheritance (SYS08) ----
/// <summary>Freezes the carry plan off the closing run: the 20%
/// floored gold, the kept trader ids (the core desk follows
/// along) and the run index the carry seeds. Read-only - the
/// world never moves here.</summary>
public NewRunCarry PlanNewRun()
{
    var carry = new NewRunCarry
    {
        GoldHc = _stage.Market.TotalAssetsHc * TradeData.InheritGoldPercent / 100,
        RunIndex = _runCount + 1,
    };
    if (_desk != null)
        foreach (var t in _desk.Collection)
            carry.TraderIds.Add(t.id);
    return carry;
}

/// <summary>Wires the next run onto the fresh stage: the carry
/// seeds the wallet, the kept traders land on the fresh desk's
/// collection (no draw, no ad burn), the seated inheritance
/// re-lives the first-trader experience (the T2 gate flag
/// re-arms), the ladder restarts, the albums and the curve
/// history persist and the ending re-arms for the new run.
/// Returns the applied carry plan.</summary>
```

```

public NewRunCarry StartNewRun(TradeStage freshStage, TradeDesk freshDesk)
{
    var carry = PlanNewRun();
    freshStage.Market.BookGoldHc(carry.GoldHc);
    if (freshDesk != null && carry.TraderIds.Count > 0)
    {
        freshDesk.InheritFrom(carry.TraderIds);
        freshDesk.SyncRoster(1);
        if (freshDesk.Roster.Count > 0)
            freshStage.Gate.RaiseFlag(TradeData.Tiers()[1].flagEventId); // the inherited seat re-lives it
    }
    _stage = freshStage;
    _desk = freshDesk;
    _runCount++;
    _endingReached = false;
    _closeSeen = 0;
    _fillSeen = 0;
    _eventSeen = 0;
    _promoSeen = 0;
    _funnel.Add("evt_new_run@" + _runCount + ":" + carry.GoldHc);
    return carry;
}

static string HcStr(long hc)
    => (hc / 100.0).ToString("0.##", CultureInfo.InvariantCulture);
/// <summary>Canonical fingerprint - the zero-RNG parity instrument
/// (the showcase ledger plus the whole stage signature).</summary>
public string Fingerprint()
{
    var sb = new System.Text.StringBuilder();
    sb.Append("run").Append(_runCount);
    sb.Append("|end").Append(_endingReached ? 1 : 0);
    sb.Append("|curve").Append(_curve.Count);
    foreach (var p in _curve) sb.Append(';').Append(p.run).Append(',').Append(p.day).Append('@').Append(p.assets);
    sb.Append("|alb").Append(_traderAlbum.Count).Append(',').Append(_stockAlbum.Count).Append(',').Append(_event);
    sb.Append("|fun").Append(_funnel.Count);
    foreach (var f in _funnel) sb.Append('#').Append(f);
    sb.Append("||").Append(_stage.OpeningSig());
    return sb.ToString();
}
}

// TradeStage.cs
// TradeStage.cs - the opening beat driver (G10-T005).
// Drives TradeMarket through the frozen red-line-11 opening script and
// stamps every promise on the sim clock: the K-line hook owns t0, the
// teaching order lands during the auction (pre-buried for the open print),
// the open bell rings at 8.00s and the fill fires the explosion triple, the
// opening bounce books the first coin within 1.5s of the fill with the
// coin-roll triple on one tick, then the all-in bro cameo (the first laugh)
// and the first market event card land inside the 10s window. Pure C#

```

```

// zero-RNG: the same script always fingerprints byte-identical.
using System.Collections.Generic;
namespace Sy.CrazyStocks
{
    public enum TradePhase
    {
        Hook,           // the K-line big screen + the limit-up red board caption
        AuctionPending, // the teaching order sits in the auction
        FirstFill,       // the open bell rang, the first fill printed
        TradingUp,       // the first coin rolled in
    }

    public sealed class TradeStage
    {
        // the frozen opening script (red-line-11 beat anchors)
        public const double OrderWindowStart = 3.0; // T+3s: the order face goes live
        public const double TraderCameoAt = 9.0;    // T+9s: the all-in bro cameo (the first laugh)
        public const double MarketEventAt = 9.5;    // T+9.5s: the first market event card
        public const string TutorialStockId = "stk_leek_power"; // the T1 single main-board meme stock

        readonly TradeMarket _market;
        readonly TradeEventEngine _events; // the T007 event stream rides the stage clock
        readonly TradeTierGate _gate;      // the T008 promotion spine (SYS05)
        readonly TradeIpoFund _ipoFund;    // the T009 tier-ticket desk (SYS07)
        readonly TradeBoard _board;        // the T013 boards face (SYS06, revisit-refresh law)
        readonly TradeContent _content;     // the T015 content trio (M16/M17/M23) rides the close bell
        readonly TradeTierContent _tierContent; // the T016 quartet (M18/M19/M24/M25) rides the close bell, tier-gated
        readonly List<(string id, double t)> _stamps = new List<(string, double)>();
        double _t;
        int _assetHighWaterHc = TradeMarket.StartGoldHc;
        int _closeSeen; // the session-mark scan pointer (the close-bell seal watch)
        bool _bellStamped;
        bool _firstFill;
        bool _firstCoin;
        bool _cameoDone;
        bool _eventDone;
        int _fillCountSeen;
        int _firstCoinAssetsBefore;

        TradeStage(TradeMarket market)
        {
            _market = market;
            _events = new TradeEventEngine(market, 1);
            _gate = new TradeTierGate();
            _ipoFund = new TradeIpoFund(market) { Gate = _gate }; // the tier-ticket desk wires onto the spine
            _board = new TradeBoard(market, _ipoFund); // the boards ride the market + the SYS07 dragon feed (the host
            _content = new TradeContent(market); // the content trio settles on the close bell (no stamp face: the ope
            _tierContent = new TradeTierContent(market); // the tier-content quartet settles on the same bell (steals
        }

        /// <summary>The fresh leek: the private stash lands in the wallet
        /// eight seconds before the opening bell, the board glows red.</summary>
        public static TradeStage NewLeek()
            => new TradeStage(new TradeMarket(TradeMarket.OpeningJoinAuctionSecond));
    }
}

```

```

// ---- live faces ----
public double SimTime => _t;
public TradePhase Phase
{
    get
    {
        if (_firstCoin) return TradePhase.TradingUp;
        if (_firstFill) return TradePhase.FirstFill;
        if (TimeOf("evt_first_order") >= 0) return TradePhase.AuctionPending;
        return TradePhase.Hook;
    }
}
public TradeMarket Market => _market;
public TradeEventEngine Events => _events;
public TradeTierGate Gate => _gate;    // the promotion spine (the desk and UI faces wire onto it)
public TradeIpoFund IpoFund => _ipoFund;    // the T009 tier-ticket desk (IPO / counter-bet / joint push / fun
public TradeBoard Board => _board;    // the T013 boards face (dragon + perf, Refresh() on every visit)
public TradeContent Content => _content;    // the T015 content trio (sadness economy / intel desk / replay th
public TradeTierContent TierContent => _tierContent;    // the T016 quartet (mystic plaque / stuck chronicle /
public bool ModalActive => false;    // zero login wall, zero blocking modal (red line 11)
public bool CanOrder => TimeOf("evt_first_order") < 0;    // the order face is live right after the hook
public IReadOnlyList<Stamp>(string id, double t)> Stamps => _stamps;
/// <summary>First stamp time of an event id (-1 when never fired).</summary>
public double TimeOf(string id)
{
    for (int i = 0; i < _stamps.Count; i++)
        if (_stamps[i].id == id) return _stamps[i].t;
    return -1;
}
void Stamp(string id) => _stamps.Add((id, _t));
/// <summary>Advances the opening clock: drives the market and
/// fires the scheduled beats (open bell, first fill, first coin,
/// trader cameo, market event card) on the frozen script.</summary>
public void Advance(double dt)
{
    if (dt <= 0) return;
    if (TimeOf("evt_trade_hook") < 0)
        Stamp("evt_trade_hook");    // the hook owns t0 (K-line screen + red board + caption)
    _t += dt;
    _market.Advance(dt);
    // the open bell crossing (the market clock owns the ring)
    if (!_bellStamped)
    {
        for (int i = 0; i < _market.SessionMarks.Count; i++)
        {
            if (_market.SessionMarks[i].sessionId == "open_bell")
            {
                Stamp("evt_open_bell");
                _bellStamped = true;
                break;
            }
        }
    }
}

```

```
    }
}
}
// the first fill: the explosion triple fires on the fill tick
if (!_firstFill && _market.Fills.Count > _fillCountSeen)
{
    _fillCountSeen = _market.Fills.Count;
    _firstFill = true;
    Stamp("evt_first_fill");
    Stamp("fill_visual"); // the fill explosion face
    Stamp("fill_sfx");    // the match chime trigger
    Stamp("fill_number"); // the wallet spend roll
}
// the first coin: the first total-assets high-water roll-up
// (the intraday peak face: the coin fires on the tick print
// that first crosses the high-water mark)
int peak = _market.MaxAssetsHc;
if (peak > _assetHighWaterHc)
{
    if (!_firstCoin)
    {
        _firstCoinAssetsBefore = _assetHighWaterHc;
        Stamp("evt_first_coin");
        Stamp("coin_visual"); // the coin-roll face
        Stamp("coin_sfx");    // the signature coin chime trigger
        Stamp("coin_number"); // the assets counter jump
        _firstCoin = true;
    }
    _assetHighWaterHc = peak;
}
// the first laugh: the all-in bro cameo with the attitude line
if (!_cameoDone && _t >= TraderCameoAt)
{
    Stamp("evt_trader_cameo");
    Stamp("evt_first_laugh");
    _cameoDone = true;
}
// the first market event card (the +2% teaching disturbance)
if (!_eventDone && _t >= MarketEventAt)
{
    _market.ApplyEventDisturbance(TutorialStockId, 2);
    Stamp("evt_market_event_card");
    _eventDone = true;
}
// the T007 event stream: the rotation slots and the chain
// beats ride the same clock (first wave at the P50 anchor)
_events.Advance(dt);
// the T008 promotion spine: the sealed-board watch first (the
// T3 lived experience), then the gate tick - a promotion
// re-scopes the event pool and stamps the ceremony beats
```

```

    ObserveCloseBellSeals();
    _gate.Advance(dt);
    if (_gate.Tick(_market.TotalAssetsHc))
    {
        _events.SetTier(_gate.Tier);
        Stamp("evt_rank_up");
        Stamp(TradeTierGate.CeremonyAwardVisual);
        Stamp(TradeTierGate.CeremonyMcBroadcast);
    }
}

/// <summary>The close-bell seal watch (the T3 lived experience):
/// a held stock sealing the limit-up board at the closing bell =
/// the player rode the squad's board. E06 stays the post-promotion
/// milestone celebration card; the flag comes from the market
/// itself, so the gate chain never deadlocks on a T3-gated card.
/// Raised once, idempotent on the flag face.</summary>
void ObserveCloseBellSeals()
{
    var marks = _market.SessionMarks;
    while (_closeSeen < marks.Count)
    {
        var m = marks[_closeSeen++];
        if (m.sessionId != "close_bell") continue;
        _ipoFund.OnCloseBell(m.day); // the bell hands the tier-ticket settlements (T009) before the seal watc
        _content.OnCloseBell(m.day); // the bell settles the content trio (T015: sadness yields + the replay t
        _tierContent.OnCloseBell(m.day, _gate.Tier); // the bell beats the tier-content quartet (T016: steles
        foreach (var def in TradeData.Stocks())
        {
            var s = _market.Stock(def.id);
            if (s == null || s.shares <= 0) continue;
            int limitBp = TradeData.BoardFor(def.boardId).limitPct * 100;
            if (_market.EffectiveDayBp(s) >= limitBp)
            {
                _gate.RaiseFlag(TradeData.Tiers()[2].flagEventId);
                return;
            }
        }
    }
}

/// <summary>The teaching order: an all-in-flavored market buy of
/// 90% of the stash on the tutorial stock (pre-buried in the
/// auction, it matches at the opening print).</summary>
public bool PlaceFirstOrder()
{
    if (TimeOf("evt_trade_hook") < 0) return false; // the hook owns t0
    if (TimeOf("evt_first_order") >= 0) return false; // one teaching order
    var s = _market.Stock(TutorialStockId);
    int budget = _market.WalletHc * TradeMarket.TutorialBuyPer mille / 1000;
    int shares = budget / s.priceHc;
    int ticket = _market.PlaceOrder(TutorialStockId, true, true, 0, shares);
}

```

```
        if (ticket &lt;= 0) return false;
        Stamp("evt_first_order");
        return true;
    }
    public int FirstCoinJumpHc => _firstCoin ? _market.MaxAssetsHc - _firstCoinAssetsBefore : 0;
    public string CameoLine => TradeTexts.EnText(TradeTexts.KeyTraderAllInBro);
    /// <summary>Opening fingerprint: every stamp plus the market state
    /// plus the event stream plus the promotion spine - byte-identical
    /// across identical scripts.</summary>
    public string OpeningSig()
    {
        var sb = new System.Text.StringBuilder();
        foreach (var s in _stamps) sb.Append(s.id).Append('@').Append(s.t.ToString("0.###")).Append(';');
        sb.Append(_market.Fingerprint());
        sb.Append("||").Append(_events.Fingerprint());
        sb.Append("||").Append(_gate.Fingerprint());
        return sb.ToString();
    }
}
}
// TradeTexts.cs
// TradeTexts.cs - the exchange copy law for Crazy Trade (G10-T004).
// English source strings live here as compiled code (the family source law:
// code strings stay English); the zh-CN localization rides a separate data
// file (Assets/_Game/Configs/Texts_zhCN.json - data, not source). Lookup
// follows the shared LocaleService contract: a missing key falls back to the
// key itself (machine-readable, never a crash), and a zh entry missing from
// the data file falls back to the English source. Zero scattered copy: every
// UI-facing sentence routes through these keys - DataSchemaCheck scans the
// runtime tree for stray CJK and proves the coverage law. The fictional-
// market disclaimer is a permanent resident key (compliance face, always on).
using System;
using System.Collections.Generic;
using Sy.Core;
namespace Sy.CrazyStocks
{
    public static class TradeTexts
    {
        // ---- key constants (the compiled copy surface) ----
        public const string KeyHookCaption = "hook_caption";
        public const string KeyMarketDisclaimer = "market_disclaimer"; // the permanent compliance face
        public const string KeyTier1 = "tier_1";
        public const string KeyTier2 = "tier_2";
        public const string KeyTier3 = "tier_3";
        public const string KeyTier4 = "tier_4";
        public const string KeyTier5 = "tier_5";
        public const string KeyStatAssets = "stat_assets";
        public const string KeyStatTier = "stat_tier";
        public const string KeyStatGold = "stat_gold";
        public const string KeyBoardMain = "board_main";
    }
}
```

```
public const string KeyBoardGem = "board_gem";
public const string KeyBoardCrazy = "board_crazy";
public const string KeyBoardNight = "board_night";
public const string KeySessionAuction = "session_auction";
public const string KeySessionOpenBell = "session_open_bell";
public const string KeySessionMorning = "session_morning";
public const string KeySessionLunch = "session_lunch";
public const string KeySessionAfternoon = "session_afternoon";
public const string KeySessionCloseBell = "session_close_bell";
public const string KeyLimitUp = "limit_up";
public const string KeyLimitDown = "limit_down";
public const string KeyOrderPlace = "order_place";
public const string KeyOrderFilled = "order_filled";
public const string KeyFeeTrade = "fee_trade";
public const string KeyIpoLuck = "ipo_luck";
public const string KeyJointBoard = "joint_board";
public const string KeyFundFee = "fund_fee";
// ---- T009 tier-ticket seats: the IPO desk, the counter-bet, the
// joint push and the fund faces (closed compiled set) ----
public const string KeyIpoApply = "ipo_apply";
public const string KeyIpoMiss = "ipo_miss";
public const string KeyIpoBroke = "ipo_broke";
public const string KeyIpoSurvive = "ipo_survive";
public const string KeyCounterBet = "counter_bet";
public const string KeyCounterWin = "counter_bet_win";
public const string KeyCounterLose = "counter_bet_lose";
public const string KeyGangUpTea = "gang_up_tea";
public const string KeyDragonList = "dragon_list";
public const string KeyFundIssue = "fund_issue";
public const string KeyFundDanmaku1 = "fund_danmaku_1";
public const string KeyFundDanmaku2 = "fund_danmaku_2";
public const string KeyFundDanmaku3 = "fund_danmaku_3";
public const string KeyIcebergDesk = "iceberg_desk";
public const string KeyBigLotSelfLift = "big_lot_self_lift";
public const string KeyIcebergLanded = "iceberg_landed";
public const string KeyIcebergBuilt = "iceberg_built";
public const string KeyRankUp = "rank_up";
public const string KeyRankUpMc = "rank_up_mc";
public const string KeyTraderAllInBro = "trader_all_in_bro";
public const string KeyTraderSteadyWang = "trader_steady_wang";
public const string KeyTraderCounterLi = "trader_counter_li";
public const string KeyTraderGrannyInfo = "trader_granny_info";
public const string KeyTraderBoardSnatcher = "trader_board_snatcher";
public const string KeyTraderTopEscapeSis = "trader_top_escape_sis";
public const string KeyTraderLeekTrainer = "trader_leek_trainer";
public const string KeyAlbumTitle = "album_title";
public const string KeyCurveTitle = "curve_title";
public const string KeyOfflineSummary = "offline_summary";
public const string KeyOfflineDouble = "offline_double";
public const string KeyOvernightGap = "overnight_gap";
```

```
public const string KeyCarryLabel = "carry_label";
public const string KeyNewJourney = "new_journey";
public const string KeyNewJourneyConfirm = "new_journey_confirm";
public const string KeyShareChallenge = "share_challenge";
public const string KeyDailyReport = "daily_report";
// ---- schema v2 seats: the 8th/9th trader cards + the E01-E20 event
// title closed set (every event row carries exactly one of these) ----
public const string KeyTraderNeverWrongDing = "trader_never_wrong_ding";
public const string KeyTraderSweeperMonk = "trader_sweeper_monk";
public const string KeyEventTitleE01 = "event_title_e01";
public const string KeyEventTitleE02 = "event_title_e02";
public const string KeyEventTitleE03 = "event_title_e03";
public const string KeyEventTitleE04 = "event_title_e04";
public const string KeyEventTitleE05 = "event_title_e05";
public const string KeyEventTitleE06 = "event_title_e06";
public const string KeyEventTitleE07 = "event_title_e07";
public const string KeyEventTitleE08 = "event_title_e08";
public const string KeyEventTitleE09 = "event_title_e09";
public const string KeyEventTitleE10 = "event_title_e10";
public const string KeyEventTitleE11 = "event_title_e11";
public const string KeyEventTitleE12 = "event_title_e12";
public const string KeyEventTitleE13 = "event_title_e13";
public const string KeyEventTitleE14 = "event_title_e14";
public const string KeyEventTitleE15 = "event_title_e15";
public const string KeyEventTitleE16 = "event_title_e16";
public const string KeyEventTitleE17 = "event_title_e17";
public const string KeyEventTitleE18 = "event_title_e18";
public const string KeyEventTitleE19 = "event_title_e19";
public const string KeyEventTitleE20 = "event_title_e20";
// ---- T011 showcase seats: the three album tabs, the crazy-index
// four-band closed set and the fund-manager summit ending ----
public const string KeyAlbumTraders = "album_traders";
public const string KeyAlbumStocks = "album_stocks";
public const string KeyAlbumEvents = "album_events";
public const string KeyCrazyFlat = "crazy_flat";
public const string KeyCrazyGreed = "crazy_greed";
public const string KeyCrazyFrenzy = "crazy_frenzy";
public const string KeyCrazyPanic = "crazy_panic";
public const string KeyEndingTitle = "ending_title";
// ---- T013 board seats: the two-board titles, the player seat,
// the card slot line and the hot-money crew signature closed set
// (the actor-troupe cameo law) ----
public const string KeyBoardTitle = "board_title";
public const string KeyBoardDragon = "board_dragon";
public const string KeyBoardPerf = "board_perf";
public const string KeyBoardYou = "board_you";
public const string KeyBoardCardLine = "board_card_line";
public const string KeyOppSquadAdmiral = "opp_squad_admiral";
public const string KeyOppFloorSky = "opp_floor_sky";
public const string KeyOppAuctionBro = "opp_auction_bro";
```

```
public const string KeyOppOneDayTourist = "opp_one_day_tourist";
// ---- T014 M15 LIVE dual-time faces: the compliance wording (the
// market calls itself the live tape, never a real exchange) and
// the three clock-mode labels ----
public const string KeyLiveTape = "live_tape";
public const string KeyClockLive = "clock_mode_live";
public const string KeyClockReplay = "clock_mode_replay";
public const string KeyClockClosed = "clock_mode_closed";
// ---- T015 content-trio seats: the three sadness assets (M16),
// the three guru schools with the fee/scam/degree faces (M17)
// and the replay theater pool (M23: the title, the six rotating
// templates, the rehearsal skip line) ----
public const string KeySadAssetNoodle = "sad_asset_noodle";
public const string KeySadAssetCafe = "sad_asset_cafe";
public const string KeySadAssetFireworks = "sad_asset_fireworks";
public const string KeySadYieldLine = "sad_yield_line";
public const string KeyIntelSchoolDance = "intel_school_dance";
public const string KeyIntelSchoolMystic = "intel_school_mystic";
public const string KeyIntelSchoolTechnical = "intel_school_technical";
public const string KeyIntelFeeLine = "intel_fee_line";
public const string KeyIntelScamEvent = "intel_scam_event";
public const string KeyAchievementTrickedThrice = "achievement_tricked_thrice";
public const string KeyReplayTitle = "replay_title";
public const string KeyReplayTpl1 = "replay_tpl_1";
public const string KeyReplayTpl2 = "replay_tpl_2";
public const string KeyReplayTpl3 = "replay_tpl_3";
public const string KeyReplayTpl4 = "replay_tpl_4";
public const string KeyReplayTpl5 = "replay_tpl_5";
public const string KeyReplayTpl6 = "replay_tpl_6";
public const string KeyReplaySkipLine = "replay_skip_line";
// the T016 tier-content quartet keys (M18/M19/M24/M25)
public const string KeyMysticSchoolFengShui = "mystic_school_feng_shui";
public const string KeyMysticSchoolTaoFlow = "mystic_school_tao_flow";
public const string KeyMysticSchoolKoiPalace = "mystic_school_koi_palace";
public const string KeyMysticFortuneFeng1 = "mystic_fortune_feng_1";
public const string KeyMysticFortuneFeng2 = "mystic_fortune_feng_2";
public const string KeyMysticFortuneFeng3 = "mystic_fortune_feng_3";
public const string KeyMysticFortuneTao1 = "mystic_fortune_tao_1";
public const string KeyMysticFortuneTao2 = "mystic_fortune_tao_2";
public const string KeyMysticFortuneTao3 = "mystic_fortune_tao_3";
public const string KeyMysticFortuneKoi1 = "mystic_fortune_koi_1";
public const string KeyMysticFortuneKoi2 = "mystic_fortune_koi_2";
public const string KeyMysticFortuneKoi3 = "mystic_fortune_koi_3";
public const string KeyMysticCompliance = "mystic_compliance";
public const string KeyUnstuckCertTitle = "unstuck_cert_title";
public const string KeyUnstuckCertStock = "unstuck_cert_stock";
public const string KeyUnstuckCertDays = "unstuck_cert_days";
public const string KeyUnstuckCertDepth = "unstuck_cert_depth";
public const string KeyUnstuckCertDay = "unstuck_cert_day";
public const string KeyLeekBronze = "leek_bronze";
```

```
public const string KeyLeekSilver = "leek_silver";
public const string KeyLeekGold = "leek_gold";
public const string KeyLeekPlatinum = "leek_platinum";
public const string KeyLeekDiamond = "leek_diamond";
public const string KeyLeekStar = "leek_star";
public const string KeyLeekKing = "leek_king";
public const string KeyLeekSupreme = "leek_supreme";
public const string KeyLeekBadgeLine = "leek_badge_line";
public const string KeyCityMoodFrenzy1 = "city_mood_frenzy_1";
public const string KeyCityMoodFrenzy2 = "city_mood_frenzy_2";
public const string KeyCityMoodFrenzy3 = "city_mood_frenzy_3";
public const string KeyCityMoodGreed1 = "city_mood_greed_1";
public const string KeyCityMoodGreed2 = "city_mood_greed_2";
public const string KeyCityMoodGreed3 = "city_mood_greed_3";
public const string KeyCityMoodPanic1 = "city_mood_panic_1";
public const string KeyCityMoodPanic2 = "city_mood_panic_2";
public const string KeyCityMoodPanic3 = "city_mood_panic_3";
public const string KeyCityMoodFlat1 = "city_mood_flat_1";
public const string KeyCityMoodFlat2 = "city_mood_flat_2";
public const string KeyCityMoodFlat3 = "city_mood_flat_3";
public const string KeyCityManiaLine = "city_mania_line";
public const string KeyManiaCompliance = "mania_compliance";
// ---- T023a M21 drama-theater seats: the blind-box face line, the
// fictional regulator letter and the th01-th08 pool title closed
// set (the letter never fines - pure theater, zero real-technique
// wording, the compliance law) ----
public const string KeyTheaterBlindBox = "theater_blind_box";
public const string KeyTheaterRegulatorLetter = "theater_regulator_letter";
public const string KeyTheaterTitleTh01 = "theater_title_th01";
public const string KeyTheaterTitleTh02 = "theater_title_th02";
public const string KeyTheaterTitleTh03 = "theater_title_th03";
public const string KeyTheaterTitleTh04 = "theater_title_th04";
public const string KeyTheaterTitleTh05 = "theater_title_th05";
public const string KeyTheaterTitleTh06 = "theater_title_th06";
public const string KeyTheaterTitleTh07 = "theater_title_th07";
public const string KeyTheaterTitleTh08 = "theater_title_th08";
// ---- T024a M22 fund-sentiment seats: the three holder-steadying
// means (the letter, the roadshow, the red packet), the tide face
// and the rt01-rt06 redemption-tide comedy pool titles (the tide
// is comedy, never a death spiral - the fund always rebuilds,
// zero real-technique wording, the compliance law) ----
public const string KeyFundLetter = "fund_letter";
public const string KeyFundRoadshow = "fund_roadshow";
public const string KeyFundRedpacket = "fund_redpacket";
public const string KeyFundTide = "fund_tide";
public const string KeyFundTideTitleRt01 = "fund_tide_title_rt01";
public const string KeyFundTideTitleRt02 = "fund_tide_title_rt02";
public const string KeyFundTideTitleRt03 = "fund_tide_title_rt03";
public const string KeyFundTideTitleRt04 = "fund_tide_title_rt04";
public const string KeyFundTideTitleRt05 = "fund_tide_title_rt05";
```

```
public const string KeyFundTideTitleRt06 = "fund_tide_title_rt06";
/// <summary>English source table (compiled - the source-of-law face).</summary>
public static readonly Dictionary<string, string> En = new Dictionary<string, string>
{
    { KeyHookCaption, "WELCOME TO CRAZY TRADE - YOUR FUND-MANAGER LIFE STARTS NOW" },
    { KeyMarketDisclaimer, "FICTIONAL MARKET FOR ENTERTAINMENT - NOT INVESTMENT ADVICE" },
    { KeyTier1, "THE LEEK RETAIL" },
    { KeyTier2, "MID-TIER" },
    { KeyTier3, "BIG PLAYER" },
    { KeyTier4, "HOT MONEY" },
    { KeyTier5, "FUND MANAGER" },
    { KeyStatAssets, "ASSETS" },
    { KeyStatTier, "TIER" },
    { KeyStatGold, "GOLD" },
    { KeyBoardMain, "MAIN BOARD" },
    { KeyBoardGem, "GEM MEME BOARD" },
    { KeyBoardCrazy, "CRAZY BOARD" },
    { KeyBoardNight, "NIGHT-SNACK BOARD" },
    { KeySessionAuction, "CALL AUCTION" },
    { KeySessionOpenBell, "OPENING BELL" },
    { KeySessionMorning, "MORNING SESSION" },
    { KeySessionLunch, "LUNCH BREAK" },
    { KeySessionAfternoon, "AFTERNOON SESSION" },
    { KeySessionCloseBell, "CLOSING BELL" },
    { KeyLimitUp, "LIMIT UP!" },
    { KeyLimitDown, "LIMIT DOWN..." },
    { KeyOrderPlace, "PLACE ORDER" },
    { KeyOrderFilled, "FILLED!" },
    { KeyFeeTrade, "FEE: {0}%" },
    { KeyIpoLuck, "ALLOTTED! RARER THAN A LOTTERY WIN" },
    { KeyJointBoard, "JOINT BOARD PUSH" },
    { KeyFundFee, "FUND FEE: {0}%/DAY" },
    { KeyIpoApply, "IPO SUBSCRIPTION - SLOTS BOOKED, PRAY FOR LUCK" },
    { KeyIpoMiss, "MISSED - 0.3% IS 0.3%" },
    { KeyIpoBroke, "BROKE BELOW ISSUE - THE HYPE PREMIUM EVAPORATES" },
    { KeyIpoSurvive, "HELD THROUGH THE BREAK - THE LEEK LIVES" },
    { KeyCounterBet, "COUNTER-BET - RIDE AGAINST THE CALL" },
    { KeyCounterWin, "THE CALL WAS WRONG AGAIN - DOUBLE THE STAKE" },
    { KeyCounterLose, "THE CALL WAS RIGHT FOR ONCE - THE STAKE IS GONE" },
    { KeyGangUpTea, "THE REGULATOR POURS TEA (PURELY FICTIONAL)" },
    { KeyDragonList, "THE DRAGON LIST - TODAY'S GANG-UP SEALS" },
    { KeyFundIssue, "ISSUE THE CRAZY FUND - THE HOLDERS ARRIVE" },
    { KeyFundDanmaku1, "PRAISE THE MANAGER - FLAT IS THE NEW UP" },
    { KeyFundDanmaku2, "MANAGER, SHOW UP - THE COMMENT SECTION MISSES YOU" },
    { KeyFundDanmaku3, "ALL-IN ON THE FUND - WHAT COULD GO WRONG" },
    { KeyIcebergDesk, "ICEBERG DESK - SLICE THE MEGALOT, SLIP IN QUIET" },
    { KeyBigLotSelfLift, "THE MEGALOT LIFTED THE PRICE AGAINST YOU - THE DESK CHUCKLES" },
    { KeyIcebergLanded, "ONE MORE QUIET SLICE LANDED - STILL UNDER THE RADAR" },
    { KeyIcebergBuilt, "POSITION BUILT BELL BY BELL - THE MARKET NEVER NOTICED" },
    { KeyRankUp, "TIER UP! {0} UNLOCKED" },
```

```
{ KeyRankUpMc, "CRAZY FINANCE CHANNEL:" },
{ KeyTraderAllInBro, "ALL-IN BRO - ALL-IN IS AN ATTITUDE" },
{ KeyTraderSteadyWang, "STEADY WANG - HALF POSITION, FULL NOON NAP" },
{ KeyTraderCounterLi, "COUNTER LI - TRUST LI, THEN DO THE OPPOSITE" },
{ KeyTraderGrannyInfo, "GRANNY INFO - THE SQUARE-DANCE WIRE" },
{ KeyTraderBoardSnatcher, "BOARD SNATCHER - LIMIT-UP OR NOTHING" },
{ KeyTraderTopEscapeSis, "TOP-ESCAPE SIS - SELLING EARLY IS ALSO PROFIT" },
{ KeyTraderLeekTrainer, "LEEK TRAINER - LOSSES BECOME EXPERIENCE" },
{ KeyAlbumTitle, "MEME ALBUM" },
{ KeyCurveTitle, "MY ASSET CURVE" },
{ KeyOfflineSummary, "WHILE YOU WERE GONE..." },
{ KeyOfflineDouble, "DOUBLE IT?" },
{ KeyOvernightGap, "OVERNIGHT GAP: {0}%" },
{ KeyCarryLabel, "NEXT-RUN CARRY: {0}" },
{ KeyNewJourney, "START A NEW RUN" },
{ KeyNewJourneyConfirm, "TAP AGAIN - THE CARRY SEEDS YOUR NEXT LEEK" },
{ KeyShareChallenge, "CAN YOUR CURVE BEAT MINE?" },
{ KeyDailyReport, "DAY {0}: {1}" },
{ KeyTraderNeverWrongDing, "NEVER-WRONG DING - IT'S NOT ME, IT'S THE MARKET" },
{ KeyTraderSweeperMonk, "THE SWEEPING MONK - I DON'T TRADE, I JUST SWEEP" },
{ KeyEventTitleE01, "METAVERSE PANCAKE WINS A TEN-MILLION FUNDING ROUND" },
{ KeyEventTitleE02, "LEEK POWER ANNUAL REPORT BLOWS UP" },
{ KeyEventTitleE03, "SQUARE-DANCE AUDIO WINS A GOVERNMENT ORDER" },
{ KeyEventTitleE04, "A HOT-MONEY WHALE DROPS BY" },
{ KeyEventTitleE05, "SYSTEM MAINTENANCE FALSE ALARM" },
{ KeyEventTitleE06, "THE LIMIT-UP SQUAD ASSEMBLES" },
{ KeyEventTitleE07, "IPO DAY - THE ALLOTMENT CEREMONY" },
{ KeyEventTitleE08, "GANG-UP SUCCESS - A SMALL MEME STOCK HITS THE BOARD" },
{ KeyEventTitleE09, "THE FUND-HOLDER COMMENT GALA" },
{ KeyEventTitleE10, "THE OVERNIGHT GAP SURPRISE" },
{ KeyEventTitleE11, "THE LIGHTS-OUT NOODLE FESTIVAL" },
{ KeyEventTitleE12, "THE ROOFTOP VIEW CAFE OPENS" },
{ KeyEventTitleE13, "THE 88TH-FLOOR SENTRY TOWER OPENS" },
{ KeyEventTitleE14, "HALFWAY MONUMENT VS FLOOR-PRICE MEMORIAL" },
{ KeyEventTitleE15, "CRAZY CIRCUIT-BREAKER DAY" },
{ KeyEventTitleE16, "THE ANALYST TRIO TV DEBATE" },
{ KeyEventTitleE17, "THE CROW-MOUTH HITS THE STREET" },
{ KeyEventTitleE18, "THE HOARDING HAMSTER REVEALED" },
{ KeyEventTitleE19, "THE MYSTERIOUS NORTHBOUND FIGURE" },
{ KeyEventTitleE20, "THE BREAK-EVEN MEMORIAL HALL" },
{ KeyAlbumTraders, "ALBUM: THE TRADER DESK" },
{ KeyAlbumStocks, "ALBUM: MEME STOCKS" },
{ KeyAlbumEvents, "ALBUM: MARKET EVENTS" },
{ KeyCrazyFlat, "CRAZY INDEX: LAID-BACK" },
{ KeyCrazyGreed, "CRAZY INDEX: GREED" },
{ KeyCrazyFrenzy, "CRAZY INDEX: FRENZY" },
{ KeyCrazyPanic, "CRAZY INDEX: PANIC" },
{ KeyEndingTitle, "FUND-MANAGER!" },
{ KeyBoardTitle, "THE CRAZY EXCHANGE BOARDS" },
{ KeyBoardDragon, "DRAGON LIST - TODAY'S TOP MOVERS" },
```

```
{ KeyBoardPerf, "PERF BOARD - TODAY'S DAY RETURNS" },
{ KeyBoardYou, "YOU - THE LEEK IN THE ARENA" },
{ KeyBoardCardLine, "TODAY: PERF RANK #{0}/{1} - TOP MOVER {2} {3}" },
{ KeyOppSquadAdmiral, "SQUAD ADMIRAL - SEE YOU ON THE BOARD" },
{ KeyOppFloorSky, "FLOOR-TO-SKY MANIAC - THE FLOOR IS MY LAUNCHPAD" },
{ KeyOppAuctionBro, "AUCTION BRO - THE OPENING PRINT OBEYS ME" },
{ KeyOppOneDayTourist, "ONE-DAY TOURIST - IN TODAY, GONE TOMORROW" },
{ KeyLiveTape, "LIVE TAPE" },
{ KeyClockLive, "LIVE - THE REAL SESSION WINDOW, SAME MOMENT EVERYWHERE" },
{ KeyClockReplay, "REPLAY - TODAY'S 30-MINUTE REHEARSAL, FREE" },
{ KeyClockClosed, "MARKET CLOSED - THE HOLIDAY STAGE IS ON" },
{ KeySadAssetNoodle, "NOODLE BAR - THE WHOLE-CITY EATS-NOODLES INDEX FUND" },
{ KeySadAssetCafe, "ROOFTOP CAFE - THE WIND-BLOWER VIEW INDEX" },
{ KeySadAssetFireworks, "FIREWORKS STAND - THE TEARS CELEBRATION FUND" },
{ KeySadYieldLine, "SADNESS YIELD: {0} {1}bp (TAPE {2}bp)" },
{ KeyIntelSchoolDance, "SQUARE-DANCE WIRE - THE MORNING SCOOP, RUMOR-GRADE" },
{ KeyIntelSchoolMystic, "DREAM-CHART MASTERS - THE MYSTIC READERS, RUMOR-GRADE" },
{ KeyIntelSchoolTechnical, "NEVER-WRONG DESK - THE TECHNICAL BUREAU, RUMOR-GRADE" },
{ KeyIntelFeelLine, "SUBSCRIPTION: {0}/DAY - ACCURACY {1} PER MILLE, ENTERTAINMENT ONLY" },
{ KeyIntelScamEvent, "THE CALL WAS A RUMOR - THE DESK EATS NOODLES, THE RUN LIVES ON" },
{ KeyAchievementTrickedThrice, "TRICKED THRICE - THE LEEK EARNS ITS DEGREE" },
{ KeyReplayTitle, "TONIGHT'S CLOSING REPLAY THEATER" },
{ KeyReplayTpl1, "TODAY THE LEEK LANDED {0} FILLS - THE CHANNEL SALUTES THE EFFORT" },
{ KeyReplayTpl2, "TOP MOVER {1} SWUNG {2}BP - THE EXPERTS ARE SPEECHLESS" },
{ KeyReplayTpl3, "THE WALLET CLOSED AT {3} - NOODLES FOR EVERYONE" },
{ KeyReplayTpl4, "SADNESS INDEX {4}BP - THE NOODLE BAR HIRES NIGHT STAFF" },
{ KeyReplayTpl5, "THE GUEST DESK SAYS: {5}" },
{ KeyReplayTpl6, "{5} - SAME TIME TOMORROW, THE TAPE NEVER SLEEPS" },
{ KeyReplaySkipLine, "REPLAY SESSION - A FREE REHEARSAL, NO FORMAL REPLAY TONIGHT" },
{ KeyMysticSchoolFengShui, "FENG-SHUI WHEEL SECT - THE PLAQUE OF SPINNING LUCK" },
{ KeyMysticSchoolTaoFlow, "TAO FLOW GATE - THE PLAQUE OF EFFORTLESS DRIFT" },
{ KeyMysticSchoolKoiPalace, "KOI PALACE - THE PLAQUE OF THE LUCKY FISH" },
{ KeyMysticFortuneFeng1, "A FINE DAY TO LIE FLAT - ORDERS FILL SLOWER, BUT THE INSOMNIA EVENT CANNOT TOUCH Y" },
{ KeyMysticFortuneFeng2, "THE WHEEL SPINS A RED TALE - SELL NOTHING, BUY NOTHING, THE UNIVERSE SHRUGS (COMED" },
{ KeyMysticFortuneFeng3, "LUCK SMELLS LIKE NOODLES TODAY - WEAR THE SEATBELT BACKWARDS, STYLE POINTS ONLY (C" },
{ KeyMysticFortuneTao1, "THE TAO SAYS DRIFT - THE TAPE WIGGLES, YOU NAP THROUGH IT (COMEDY ONLY, ZERO EDGE)" },
{ KeyMysticFortuneTao2, "FLOW LIKE WATER, TRADE LIKE A SNOOZING CAT - THE MARKET OWES YOU NOTHING, THE PLAQU" },
{ KeyMysticFortuneTao3, "TODAY'S LUCKY DIRECTION IS DOWN THE STREET FOR NOODLES - THE FORTUNE NEVER TOUCHES" },
{ KeyMysticFortuneKoi1, "THE KOI NODS ONCE - SELLING FEARS YOU TODAY, BUYING FEARS YOU MORE (COMEDY ONLY, ZE" },
{ KeyMysticFortuneKoi2, "A QUARTER MOON OVER THE EXCHANGE - WEAR SOCKS THAT CLASH, FORTUNE FOLLOWS THE BOLD" },
{ KeyMysticFortuneKoi3, "THE LUCKY FISH BLOWS A BUBBLE AT THE TICKER - IT MEANS NOTHING AND THAT IS THE JOKE" },
{ KeyMysticCompliance, "MYSTIC PLAZA IS PURE COMEDY - IT NEVER TOUCHES BUY OR SELL OUTCOMES" },
{ KeyUnstuckCertTitle, "BREAK-EVEN!" },
{ KeyUnstuckCertStock, "{0}" },
{ KeyUnstuckCertDays, "STUCK FOR {0} DAYS" },
{ KeyUnstuckCertDepth, "DEEPEST DIVE: {0}BP" },
{ KeyUnstuckCertDay, "CLAWED BACK ON DAY {0}" },
{ KeyLeekBronze, "BRONZE LEEK" },
{ KeyLeekSilver, "SILVER LEEK" },
{ KeyLeekGold, "GOLD LEEK" },
```

```

    { KeyLeekPlatinum, "PLATINUM LEEK" },
    { KeyLeekDiamond, "DIAMOND LEEK" },
    { KeyLeekStar, "STAR LEEK" },
    { KeyLeekKing, "KING LEEK" },
    { KeyLeekSupreme, "SUPREME LEEK" },
    { KeyLeekBadgeLine, "LEEK RANK: {0}" },
    { KeyCityMoodFrenzy1, "NPC: THE SQUID VENDOR QUIT SELLING SQUID AND BOUGHT THREE BOARDS" },
    { KeyCityMoodFrenzy2, "NPC: THE GRANNY DANCE TROUPE CHANGED THEIR ANTHEM TO THE TICKER SONG" },
    { KeyCityMoodFrenzy3, "NPC: THE NOODLE BAR PUT UP A FULL HOUSE SIGN AT DAWN" },
    { KeyCityMoodGreed1, "NPC: THE BARBER ASKS EVERY CLIENT ONE QUESTION - ANY TIPS" },
    { KeyCityMoodGreed2, "NPC: THE TAXI FLEET REROUTED EVERY FARE PAST THE EXCHANGE" },
    { KeyCityMoodGreed3, "NPC: THE BUBBLE TEA SHOP SELLS A LIMIT-UP FLAVOR" },
    { KeyCityMoodPanic1, "NPC: THE SQUID VENDOR IS BACK - BUSINESS IS BOOMING AGAIN" },
    { KeyCityMoodPanic2, "NPC: THE ROOFTOP CAFE RUNS OUT OF SEATS WITH A VIEW" },
    { KeyCityMoodPanic3, "NPC: THE FIREWORKS STAND RESTOCKED THREE TIMES BY NOON" },
    { KeyCityMoodFlat1, "NPC: THE SWEEPING MONK SWEEPED THE SAME SQUARE TWICE - NOTHING HAPPENED" },
    { KeyCityMoodFlat2, "NPC: THE PIGEONS FILED A COMPLAINT ABOUT THE BORING TAPE" },
    { KeyCityMoodFlat3, "NPC: THE NOODLE BAR SELLS THE USUAL - THE USUAL IS FINE" },
    { KeyCityManiaLine, "THE CRAZY GAUGE HITS FULL - THE WHOLE CITY DANCES AT ONCE" },
    { KeyManiaCompliance, "THE CITY MOOD IS PURE THEATER - NEVER AN INVESTMENT REFERENCE" },
    { KeyTheaterBlindBox, "DRAMA THEATER - EVERY MISS ROLLS A COMEDY BLIND BOX" },
    { KeyTheaterRegulatorLetter, "REGULATOR WANG'S LETTER (FICTIONAL) - TEA AND MELON SEEDS, ZERO FINE" },
    { KeyTheaterTitleTh01, "REGULATOR WANG SENDS A FICTIONAL WARNING LETTER" },
    { KeyTheaterTitleTh02, "THE HOT-MONEY CREW COUNTER-SNIPES THE PUSH" },
    { KeyTheaterTitleTh03, "THE MELON-EATING CROWD GATHERS AROUND THE TAPE" },
    { KeyTheaterTitleTh04, "THE PUSH MISSES BUT THE FAME LANDS ANYWAY" },
    { KeyTheaterTitleTh05, "THE FORUM LIGHTS UP IN A FLAME WAR" },
    { KeyTheaterTitleTh06, "THE SQUARE-DANCE TROUPE CASCADES INTO THE LOBBY" },
    { KeyTheaterTitleTh07, "THE NOODLE BAR SERVES THE SPECIAL COMBO" },
    { KeyTheaterTitleTh08, "CRAZY FINANCE CHANNEL AIRS A SPECIAL REPORT" },
    { KeyFundLetter, "REASSURANCE LETTER - THE MANAGER PINKY PROMISES" },
    { KeyFundRoadshow, "LIVESTREAM ROADSHOW - TRUST THE PROCESS, IGNORE THE CHART" },
    { KeyFundRedpacket, "GOLD RED PACKET - SORRY IN COINS, TIDE STOPPED INSTANTLY" },
    { KeyFundTide, "REDEMPTION TIDE - THE HOLDERS MARCH OUT (COMEDY, NEVER A CRASH)" },
    { KeyFundTideTitleRt01, "THE HOLDERS EXIT UNDER A FAREWELL BANNER" },
    { KeyFundTideTitleRt02, "THE FORUM THREAD IS A REDEMPTION EXPRESS" },
    { KeyFundTideTitleRt03, "THE LEEK TRAINER CALLS IT A STRATEGIC WITHDRAWAL" },
    { KeyFundTideTitleRt04, "THE BANK QUEUE STRETCHES TO THE NOODLE BAR" },
    { KeyFundTideTitleRt05, "CRAZY CHANNEL AIRS: WHERE DID THE HOLDERS GO" },
    { KeyFundTideTitleRt06, "THE SQUARE-DANCE TROUPE EXITS IN PARADE FORMATION" },
};

/// <summary>English lookup (missing key falls back to the key itself
/// - machine-readable, never a crash).</summary>
public static string EnText(string key)
    => En.TryGetValue(key, out var v) && !string.IsNullOrEmpty(v) ? v : key;
/// <summary>Parses the zh-CN data file payload into a key-to-text
/// map (values must be non-empty strings; anything else drops the
/// entry so the lookup degrades to English, never to garbage).</summary>
public static Dictionary<string, string> ParseZh(string json)
{

```

```

        var map = new Dictionary<string, string>();
        if (string.IsNullOrEmpty(json)) return map;
        try
        {
            if (!(MiniJson.Deserialize(json) is Dictionary<string, object> d)) return map;
            foreach (var kv in d)
            {
                if (!string.IsNullOrEmpty(kv.Key) && kv.Value is string s && !string.IsNullOrEmpty(s))
                {
                    map[kv.Key] = s;
                }
            }
        }
        catch
        {
            // corrupt data file = empty map = English fallback face
            return new Dictionary<string, string>();
        }
        return map;
    }

    /// <summary>LocaleService-compatible lookup delegate over a parsed
    /// zh map: zh-CN hits read the data file, everything else (and any
    /// miss) reads the English source. Wire-ready for the T005 UI face.</summary>
    public static Func<string, string, string> LookupWithZh(Dictionary<string, string> zh)
    {
        return (lang, key) =>
        {
            if (lang == "zh-CN" && zh != null && zh.TryGetValue(key, out var v) && !string.IsNullOrEmpty(v))
            {
                return v;
            }
            return EnText(key);
        };
    }
}

// TradeTierContent.cs
// TradeTierContent.cs - the T016 tier-gated content quartet (M18/M19/M24/
// M25): the born-sign mystic plaque, the stuck chronicle, the leek rank
// ladder and the city-mania dashboard upgrade. Four light mechanisms
// riding the existing faces (the halving law): the close bell hands the
// daily beat exactly like the content trio, the copy rides the M11 text
// engine (compiled English source + zh data file), and every value
// derives zero-RNG off the frozen wave math (the same day always prints
// the same numbers).
// M18 mystic plaque: the player hangs one mystic school in the hall -
// each day prints one fortune line with a nonsense comedy buff baked
// in ("a fine day to lie flat: orders fill slower, but the insomnia
// event cannot touch you"). Comedy only, zero numeric edge: the
// plaque stamps a telemetry line and touches no price, no wallet,
// no order - the compliance note is compiled dead into the copy.
// M19 stuck chronicle: every held position under its cost base gets
// a stele carved (stock id + first-stuck day + running day count +
// depth in bp); the day the position claws back over its cost the
// stele moves into the break-even memorial (the M12 face as data),
// the whole city fireworks, and the unstuck certificate composes

```

```
// as a shareable card (the F4 canary face).
// M24 leek rank: a pure-derivation ladder of self-mocking titles
// (bronze leek .. supreme leek) off the behavior score (fills +
// scam losses + break-even memorials) - titles only, zero numeric
// gain, the badge line feeds the share-card stock.
// M25 mania dashboard: the T011 crazy index upgraded into the whole
// city's mood stage - the frozen four-band set stays, each band
// rotates three NPC mood lines zero-RNG by day, and a full gauge
// (100/100) fires the city-wide mania event. Pure theater, never
// a stock tip - the compliance note is compiled dead.
using System;
using System.Collections.Generic;
using System.Text;
using Sy.Social;
namespace Sy.CrazyStocks
{
    public sealed class TradeTierContent
    {
        // ---- v1 wiring (sweep-tunable; the doc pack owns the faces) ----
        public const int MysticSchoolCount = 3;
        public const int MysticFortunePool = 3;           // three fortune lines per school, zero-RNG day rotation
        public const int UnstuckMemorialCap = 30;         // the break-even memorial hall keeps the recent stones
        public const int ManiaFullPct = 100;             // the full gauge fires the city event
        public const int LeekScamWeight = 5;             // every rumor-school scam feeds the leek score
        public const int LeekMemorialWeight = 10;        // every unstuck stone feeds the leek score
        public const string SchoolFengShuiId = "feng_shui_fortune";
        public const string SchoolTaoFlowId = "tao_flow_gate";
        public const string SchoolKoiPalaceId = "koi_palace";
        // the leek ladder (M24): frozen thresholds - pure titles, zero numeric gain
        public static readonly int[] LeekThresholds = { 0, 30, 80, 180, 400, 800, 1600, 3200 };
        public static readonly string[] LeekTitleKeys =
        {
            TradeTexts.KeyLeekBronze, TradeTexts.KeyLeekSilver, TradeTexts.KeyLeekGold, TradeTexts.KeyLeekPlatinum,
            TradeTexts.KeyLeekDiamond, TradeTexts.KeyLeekStar, TradeTexts.KeyLeekKing, TradeTexts.KeyLeekSupreme,
        };
        /// <summary>A mystic school (M18): the hall plaque - three fortune
        /// lines with the comedy buff baked in, zero-RNG rotation by day.
        /// Comedy only, zero numeric edge (the compliance note is dead
        /// law: the plaque never touches buy or sell outcomes).</summary>
        public sealed class MysticSchoolDef
        {
            public string id = "";
            public string nameKey = "";
            public int seed;
            public string[] fortuneKeys = new string[MysticFortunePool];
        }
        /// <summary>A stuck stele (M19): carved on the first close under
        /// the cost base, the day count grows each closing bell and the
        /// depth keeps the deepest underwater print seen so far (in bp)
        /// - the stone never softens a dive it already recorded.</summary>
```

```
public sealed class StuckStele
{
    public string stockId = "";
    public int sinceDay;
    public int days;
    public int depthBp;
}
/// <summary>A break-even memorial stone (M19): the stele that
/// clawed back over its cost base - the M12 memorial hall as a
/// data face (the unstuck certificate composes off it).</summary>
public sealed class UnstuckMemorial
{
    public string stockId = "";
    public int days;
    public int depthBp;
    public int day;
}
static readonly MysticSchoolDef[] MysticTable =
{
    new MysticSchoolDef
    {
        id = SchoolFengShuiId, nameKey = TradeTexts.KeyMysticSchoolFengShui, seed = 99101,
        fortuneKeys = new[] { TradeTexts.KeyMysticFortuneFeng1, TradeTexts.KeyMysticFortuneFeng2, TradeTexts.Key
    },
    new MysticSchoolDef
    {
        id = SchoolTaoFlowId, nameKey = TradeTexts.KeyMysticSchoolTaoFlow, seed = 99107,
        fortuneKeys = new[] { TradeTexts.KeyMysticFortuneTao1, TradeTexts.KeyMysticFortuneTao2, TradeTexts.KeyMy
    },
    new MysticSchoolDef
    {
        id = SchoolKoiPalaceId, nameKey = TradeTexts.KeyMysticSchoolKoiPalace, seed = 99113,
        fortuneKeys = new[] { TradeTexts.KeyMysticFortuneKoi1, TradeTexts.KeyMysticFortuneKoi2, TradeTexts.KeyMy
    },
};
readonly TradeMarket _market;
readonly List<string> _funnel = new List<string>();
readonly Dictionary<string, StuckStele> _steles = new Dictionary<string, StuckStele>();
readonly List<UnstuckMemorial> _memorial = new List<UnstuckMemorial>();
string _schoolId = "";
public TradeTierContent(TradeMarket market)
{
    _market = market;
}
// ---- live faces ----
public IReadOnlyList<string> Funnel => _funnel;
public IReadOnlyDictionary<string, StuckStele> Steles => _steles;
public IReadOnlyList<UnstuckMemorial> Memorial => _memorial;
public string SelectedSchool => _schoolId;
public static IReadOnlyList<MysticSchoolDef> MysticSchools => MysticTable;
```

```
public static MysticSchoolDef MysticSchool(string id)
{
    foreach (var s in MysticTable) if (s.id == id) return s;
    return null;
}
// ---- M18: the born-sign mystic plaque ----
/// <summary>Hangs one school's plaque in the hall (M18). The
/// plaque stamps nothing but its own face: no price, no wallet,
/// no order ever moves here (the compliance note is dead law -
/// the mystic never touches buy or sell outcomes).</summary>
public bool SelectSchool(string id)
{
    if (MysticSchool(id) == null) return false;
    _schoolId = id;
    Book("evt_mystic_plaque", id, "hung");
    return true;
}
/// <summary>The day's fortune line key (zero-RNG pool rotation):
/// the same school on the same day always prints the same line.
/// The comedy buff is baked into the copy (net-zero theater -
/// "orders fill slower, but the insomnia event cannot touch you"
/// is the joke, never a stat).</summary>
public static string FortuneKeyOf(string schoolId, int day)
{
    var s = MysticSchool(schoolId);
    if (s == null) return "";
    return s.fortuneKeys[(int)(TradeData.MixPublic(s.seed, (uint)day) % MysticFortunePool)];
}
/// <summary>The composed fortune line (empty until a plaque
/// hangs; the View reads this on the close-bell beat).</summary>
public string DailyFortuneLine(int day)
{
    if (string.IsNullOrEmpty(_schoolId)) return "";
    string key = FortuneKeyOf(_schoolId, day);
    string line = TradeTexts.EnText(key);
    Book("evt_mystic_fortune", _schoolId, "d" + day + ":" + key);
    return line;
}
// ---- M19: the stuck chronicle ----
/// <summary>The stuck law: a held position under its cost base
/// is stuck (value = shares x last print against the position
/// cost base). The depth is the underwater percentage in bp.</summary>
public static int StuckDepthBp(int shares, int priceHc, int costHc)
{
    if (shares <= 0 || costHc <= 0) return 0;
    long value = (long)shares * priceHc;
    long cost = costHc;
    if (value >= cost) return 0;
    return (int)((cost - value) * 10000 / cost);
}
```

```
/// &lt;summary&gt;The close-bell beat (wired by TradeStage exactly like
/// the content trio): the stuck chronicle scans every held
/// position (tier T2+, first stuck carve starts the chronicle),
/// the mystic plaque prints the day's fortune and the mania
/// dashboard rolls the city mood (both tier T1+). The M24 leek
/// rank is a pure derivation - no bell face, the View reads it
/// any time.&lt;/summary&gt;
public void OnCloseBell(int day, int tier)
{
    if (tier &gt;= 2)
    {
        foreach (var def in TradeData.Stocks())
        {
            var s = _market.Stock(def.id);
            if (s == null || s.shares &lt;= 0 || s.costHc &lt;= 0) continue;
            int depth = StuckDepthBp(s.shares, s.priceHc, s.costHc);
            if (depth &gt; 0)
            {
                if (!_steles.TryGetValue(def.id, out var stele))
                {
                    stele = new StuckStele { stockId = def.id, sinceDay = day, days = 1, depthBp = depth };
                    _steles[def.id] = stele;
                    Book("evt_stuck_carved", def.id, "first:d" + day + ":" + depth + "bp");
                }
                else
                {
                    stele.days++;
                    if (depth &gt; stele.depthBp) stele.depthBp = depth;    // the stone keeps the deepest print
                    Book("evt_stuck_carved", def.id, "d" + day + ":" + stele.days + "d:" + depth + "bp");
                }
            }
            else if (_steles.TryGetValue(def.id, out var stele))
            {
                // the claw-back day: the stele moves into the memorial,
                // the whole city fires fireworks, the certificate arms
                _steles.Remove(def.id);
                var stone = new UnstuckMemorial { stockId = def.id, days = stele.days, depthBp = stele.depthBp,
                _memorial.Add(stone);
                if (_memorial.Count &gt; UnstuckMemorialCap) _memorial.RemoveAt(0);
                Book("evt_unstuck_fireworks", def.id, "d" + day + ":" + stele.days + "d:" + stele.depthBp + "bp"
            }
        }
    }
    if (tier &gt;= 1)
    {
        if (!string.IsNullOrEmpty(_schoolId))
            DailyFortuneLine(day);
        var face = CrazyIndexFace();
        string moodKey = CityMoodKeyOf(face.bandKey, day);
        Book("evt_city_mood", face.bandKey, "d" + day + ":" + moodKey + ":" + face.pct);
    }
}
```

```

        if (face.pct >= ManiaFullPct)
            Book("evt_city_mania", "pct" + ManiaFullPct, "d" + day + ":" + face.netBp);
    }
}

/// <summary>The unstuck certificate card (M19): the break-even
/// memorial composes into a shareable card - the stock, the
/// stuck day count, the deepest underwater print and the
/// claw-back day (the F4 canary face). AutoCardComposer reuse,
/// no formation layer - the certificate is a copy face.</summary>
public ShareCard ComposeUnstuckCard(UnstuckMemorial stone)
{
    if (stone == null) return null;
    var lines = new List<string>
    {
        string.Format(TradeTexts.EnText(TradeTexts.KeyUnstuckCertStock), stone.stockId),
        string.Format(TradeTexts.EnText(TradeTexts.KeyUnstuckCertDays), stone.days),
        string.Format(TradeTexts.EnText(TradeTexts.KeyUnstuckCertDepth), stone.depthBp),
        string.Format(TradeTexts.EnText(TradeTexts.KeyUnstuckCertDay), stone.day),
        LeekBadgeLine(),
    };
    return AutoCardComposer.Card(TradeShowcase.CardTemplateStem,
        TradeTexts.EnText(TradeTexts.KeyUnstuckCertTitle),
        lines.ToArray(),
        TradeTexts.EnText(TradeTexts.KeyShareChallenge),
        "g10&kind=unstuck&days=" + stone.days + "&depth=" + stone.depthBp + "&day=" + stone.day)
}

// ---- M24: the leek rank ladder ----
/// <summary>The behavior score (pure derivation, zero state):
/// the fill count plus the scam losses and the break-even
/// memorials - every number re-derives off the books, so the
/// ladder can never drift from the actual play.</summary>
public int LeekScore(int scamTimes)
    => _market.Fills.Count + scamTimes * LeekScamWeight + _memorial.Count * LeekMemorialWeight;
/// <summary>The rank index off the frozen threshold table (0 =
/// bronze leek .. 7 = supreme leek). Titles only - the ladder
/// hands out no numeric gain (the compliance law of M24).</summary>
public static int LeekRankIndex(int score)
{
    int idx = 0;
    for (int i = 0; i < LeekThresholds.Length; i++)
        if (score >= LeekThresholds[i]) idx = i;
    return idx;
}

public static string LeekTitleKeyOf(int idx)
    => LeekTitleKeys[idx];
/// <summary>The badge line the share-card stock consumes (M24:
/// the badge is card material, the ladder never touches the
/// game numbers).</summary>
public string LeekBadgeLine()
    => string.Format(TradeTexts.EnText(TradeTexts.KeyLeekBadgeLine), TradeTexts.EnText(LeekTitleKeyOf(LeekRan

```

```
// ---- M25: the city-mania dashboard upgrade ----
/// &summary>The whole market's net effective day move (bp) - the
/// same sum the T011 crazy index folds (zero RNG, the same
/// clock always prints the same gauge).&summary>
public int MarketNetBp()
{
    long net = 0;
    foreach (var def in TradeData.Stocks())
    {
        var s = _market.Stock(def.id);
        if (s == null) continue;
        net += _market.EffectiveDayBp(s);
    }
    return (int)net;
}
/// &summary>The dashboard face: the net, the 0-100 gauge and the
/// frozen four-band key (the T011 law reused verbatim - the
/// upgrade adds the stage, never rewrites the bands).&summary>
public (int netBp, int pct, string bandKey) CrazyIndexFace()
{
    int net = MarketNetBp();
    return (net, TradeShowcase.CrazyIndexPctOf(net), TradeShowcase.CrazyBandKeyOf(net));
}
/// &summary>The city mood line key for a band on a day (M25):
/// three NPC mood lines per band, zero-RNG rotation by day -
/// the same city on the same day always speaks the same mood.&summary>
public static string CityMoodKeyOf(string bandKey, int day)
{
    string[] pool;
    if (bandKey == TradeTexts.KeyCrazyFrenzy) pool = new[] { TradeTexts.KeyCityMoodFrenzy1, TradeTexts.KeyCityMo
    else if (bandKey == TradeTexts.KeyCrazyGreed) pool = new[] { TradeTexts.KeyCityMoodGreed1, TradeTexts.KeyCit
    else if (bandKey == TradeTexts.KeyCrazyPanic) pool = new[] { TradeTexts.KeyCityMoodPanic1, TradeTexts.KeyCit
    else pool = new[] { TradeTexts.KeyCityMoodFlat1, TradeTexts.KeyCityMoodFlat2, TradeTexts.KeyCityMoodFlat3 };
    return pool[(int)(TradeData.MixPublic(55501, (uint)day) % pool.Length)];
}
/// &summary>Canonical fingerprint - the zero-RNG parity
/// instrument for the tier-content stream (the same script
/// over the same days is always byte-identical).&summary>
public string Fingerprint()
{
    var sb = new StringBuilder();
    sb.Append("school").Append(_schoolId);
    foreach (var f in _funnel) sb.Append(';').Append(f);
    foreach (var kv in _steles) sb.Append("|stele:").Append(kv.Key).Append('=')
        .Append(kv.Value.sinceDay).Append('/').Append(kv.Value.days).Append('/').Append(kv.Value.depthBp);
    foreach (var m in _memorial) sb.Append("|stone:").Append(m.stockId).Append('=')
        .Append(m.days).Append('/').Append(m.depthBp).Append('@').Append(m.day);
    return sb.ToString();
}
void Book(string evt, string id, string slice)
```

```

        {
            _funnel.Add(evt + "@" + id + ":" + slice);
        }
    }
}

// TradeTierGate.cs
// TradeTierGate.cs - the five-tier promotion spine (G10-T008, SYS05).
// The M04 growth spine consumes the frozen TradeData tier table: every tier
// is a dual-track gate - the meme asset threshold AND a lived-experience
// flag together, because pure money never buys a tier (a tier is a new
// gameplay ticket, never a numeric wall; the idle face unlocks on arrival,
// never blocks). The gate rises at most one tier per tick so every
// promotion owns its pixel award ceremony - the award stage, the fanfare
// and the Crazy Finance Channel broadcast each book their own beat, and
// the evt_rank_up telemetry line lands with the tier and unlock slices.
// The flag sources are the owning systems: the desk raises the
// first-trader flag on the first hire (T006), the stage raises the
// limit-up squad flag when a held stock seals the board at the bell (the
// lived T3 experience; E06 stays the post-promotion milestone card), and
// the IPO / joint-push systems raise the T4/T5 flags when they land
// (T009). The new-run reset is the inheritance law: the spine restarts
// at the leek retail tier with every flag cleared (only the opening
// stake re-arms). Pure C# zero-RNG: the same asset timeline and the same
// flags always promote on the same ticks.
using System.Collections.Generic;
using System.Text;
namespace Sy.CrazyStocks
{
    public sealed class TradeTierGate
    {
        // the ceremony beats (the pixel award show the UI face consumes)
        public const string CeremonyAwardVisual = "award_visual";
        public const string CeremonyAwardSfx = "award_sfx";
        public const string CeremonyMcBroadcast = "mc_broadcast";
        static readonly string[] CeremonyBeats = { CeremonyAwardVisual, CeremonyAwardSfx, CeremonyMcBroadcast };
        readonly List<string> _funnel = new List<string>();
        readonly List<string> _beats = new List<string>();
        readonly HashSet<string> _flags = new HashSet<string>();
        readonly List<(int tier, double at)> _promotions = new List<(int tier, double at)>();
        int _tier = 1;
        double _t;
        public TradeTierGate()
        {
            // T1 is held from the opening stake: the flag row of the first
            // gate reads the frozen table (zero drift by construction)
            _flags.Add(TradeData.Tiers()[0].flagEventId);
        }
        // ---- live faces ----
        public int Tier => _tier;
        public double SimTime => _t;
    }
}

```

```

public IReadOnlyList<string> Funnel => _funnel;
public IReadOnlyList<string> Beats => _beats;
public IReadOnlyList<(int tier, double at)> Promotions => _promotions;
public int FlagCount => _flags.Count;
public bool FlagOf(string flagEventId) => _flags.Contains(flagEventId);
/// <summary>Advances the spine clock (promotion stamps ride it).</summary>
public void Advance(double dt)
{
    if (dt > 0) _t += dt;
}
/// <summary>Raises a lived-experience flag (the owning systems
/// call this: the desk hire, the sealed board, the IPO face).
/// Idempotent - an experience counts once, whatever re-triggers it.</summary>
public void RaiseFlag(string flagEventId)
{
    if (!string.IsNullOrEmpty(flagEventId)) _flags.Add(flagEventId);
}
/// <summary>The pure five-gate solve: the highest tier whose whole
/// chain is satisfied - every asset threshold met in gold hundredths
/// AND every flag lived, walking the frozen table in order (a broken
/// link caps the ladder; the chain is monotonic by freeze). Money
/// alone never solves a gate whose flag is missing.</summary>
public static int SolveTier(long assetsHc, IEnumerable<string> flags)
{
    var lived = flags as ICollection<string> ?? new HashSet<string>(flags);
    int solved = 1;
    foreach (var def in TradeData.Tiers())
    {
        bool assetsOk = assetsHc >= def.assetGate * 100;    // gold -> hundredths
        bool flagOk = lived.Contains(def.flagEventId);
        if (assetsOk && flagOk) solved = def.tier;
        else break;
    }
    return solved;
}
/// <summary>The live solve over this gate's flags.</summary>
public int SolvedTier(long assetsHc) => SolveTier(assetsHc, _flags);
/// <summary>The promotion tick: rises at most one tier per call
/// (every promotion owns its own ceremony beat - a 100x asset
/// jump cascades over consecutive ticks, never stacks), books the
/// evt_rank_up telemetry line plus the channel broadcast line, and
/// never demotes on a drawdown. Returns true when a promotion
/// fired on this tick.</summary>
public bool Tick(long assetsHc)
{
    int next = _tier + 1;
    if (next > TradeData.TierCount) return false;
    var def = TradeData.Tiers()[next - 1];
    if (assetsHc < def.assetGate * 100) return false;    // the money track
    if (!_flags.Contains(def.flagEventId)) return false;    // the lived track
}

```

```

        _tier = next;
        _promotions.Add((next, _t));
        _funnel.Add("evt_rank_up@t" + next + ":" + def.unlockId);
        _funnel.Add("evt_rank_up_mc@t" + next);
        foreach (var b in CeremonyBeats) _beats.Add(b + "@t" + next);
        return true;
    }

    /// <summary>The tier title key (the ceremony copy face).</summary>
    public static string TierTitleKey(int tier)
    {
        switch (tier)
        {
            case 1: return TradeTexts.KeyTier1;
            case 2: return TradeTexts.KeyTier2;
            case 3: return TradeTexts.KeyTier3;
            case 4: return TradeTexts.KeyTier4;
            default: return TradeTexts.KeyTier5;
        }
    }

    /// <summary>The broadcast sentence the MC face reads on a
    /// promotion: the channel banner plus the tier-up line over the
    /// tier title (compiled English copy, zero CJK by law).</summary>
    public static string BroadcastLineOf(int tier)
        => TradeTexts.EnText(TradeTexts.KeyRankUpMc) + " "
            + string.Format(TradeTexts.EnText(TradeTexts.KeyRankUp), TradeTexts.EnText(TierTitleKey(tier)));

    /// <summary>The new-run reset (the inheritance law): the spine
    /// restarts at the leek retail tier with every flag cleared - only
    /// the opening stake re-arms (the albums and the curve history
    /// persist elsewhere, the spine never carries over).</summary>
    public void ResetForNewRun()
    {
        _tier = 1;
        _flags.Clear();
        _flags.Add(TradeData.Tiers()[0].flagEventId);
    }

    /// <summary>Canonical fingerprint - the zero-RNG parity instrument
    /// for the spine (rides the stage opening signature).</summary>
    public string Fingerprint()
    {
        var sb = new StringBuilder();
        sb.Append("tier").Append(_tier)
            .Append("|flags").Append(_flags.Count)
            .Append("|prom").Append(_promotions.Count)
            .Append("|beats").Append(_beats.Count)
            .Append("|fun").Append(_funnel.Count);
        foreach (var p in _promotions) sb.Append(";t").Append(p.tier).Append('@').Append(p.at.ToString("0.###"));
        return sb.ToString();
    }
}

```